

Nombre de la Página: ShadowAngels

Cliente: María de los Ángeles Pedraza

Equipo 7:

Diego Efraín Hernández Hernández

Omar Velázquez Pérez

Luis Ángel Roque Valdivieso

Tecnologías de Información e Innovación Digital

Desarrollo de Aplicaciones Web

Docente: Diana Xóchitl Ruano Vargas

Fecha: 06/02/26

Evidencia de autorización

La que suscribe María de los Angeles Pedraza Ortega, siendo el día 02 de Febrero del 2026 estoy conforme con la creación de la aplicación web orientada a publicidad llamada

Shadow Angels



María de los Angeles
Pedraza Ortega

Índice

Introducción	5
Descripción del cliente y problemática	6
Objetivo General y Objetivos Específicos.....	8
Alcance	9
Restricciones y Supuestos	11
Usuarios del Sistema	13
Historias de Usuario	16
Diagramas.....	20
Wireframes	24
Estructura Inicial del Frontend	33
Conclusión	55

Introducción

El presente documento corresponde a la documentación inicial del proyecto web denominado ShadowAngels, elaborado por el Equipo 7 en el marco de la materia de Desarrollo de Aplicaciones Web, perteneciente a la carrera de Tecnologías de Información e Innovación Digital de la Universidad Autónoma del Estado de Hidalgo.

El proyecto surge a partir de la necesidad identificada de una clienta real, la joven María de los Ángeles Pedraza Ortega, quien junto a su pareja gestiona un negocio informal de reventa de ropa de etiqueta y que actualmente carece de una presencia digital formal que le permita dar a conocer sus productos de manera eficiente.

El objetivo de este documento es establecer las bases del sistema a desarrollar, definiendo con claridad el alcance del proyecto, los perfiles de usuario, las funcionalidades esperadas y la estructura inicial propuesta para el frontend, todo ello sustentado en entrevistas realizadas directamente con la clienta y validado mediante carta de autorización firmada.

El sistema a desarrollar es una aplicación web de carácter publicitario, es decir, no contempla funciones de comercio electrónico como pagos en línea o gestión de entregas, sino que su propósito principal es servir como vitrina digital del negocio, permitiendo a los clientes explorar el inventario disponible y a los administradores gestionar dicho contenido de forma sencilla e intuitiva.

El stack tecnológico definido para el desarrollo del proyecto comprende Angular y Bootstrap para el frontend, Python con Flask para el backend y MongoDB como sistema de gestión de base de datos, tecnologías establecidas como requisito de la materia y acordadas con el equipo de desarrollo.

Descripción del cliente y problemática

Perfil del cliente

La clienta es la joven María de los Ángeles Pedraza Ortega, estudiante activa de la Universidad Autónoma del Estado de Hidalgo quien, junto a su pareja, gestiona un negocio informal de reventa de ropa de etiqueta. El negocio opera de manera física los días domingo en tianguis de la localidad, sin embargo, el resto de la semana las ventas y la comunicación con clientes se realizan a través de WhatsApp, plataforma mediante la cual muestran productos disponibles, acuerdan puntos de encuentro y gestionan pedidos de manera directa.

Problemática identificada

A partir de las entrevistas realizadas con la clienta, se identificó que el principal problema del negocio es la ausencia de una presencia digital formal. Actualmente, la estrategia de publicidad del negocio consiste en la difusión oral entre familiares y amigos, apoyada por el envío de fotografías tomadas desde el teléfono celular, lo cual resulta limitado, informal y en ocasiones incómodo tanto para los vendedores como para los posibles clientes.

Si bien el uso de WhatsApp ha funcionado como canal de comunicación y ventas, esta plataforma no fue diseñada para funcionar como catálogo de productos, por lo que el negocio carece de un espacio centralizado donde los clientes puedan consultar el inventario disponible de forma autónoma, sin necesidad de contactar directamente a los vendedores para obtener información básica como precios, tallas o fotografías de los productos.

Solución propuesta

Con base en la problemática descrita, a petición de la clienta se propone el desarrollo de una aplicación web de carácter publicitario denominada **ShadowAngels**, nombre definido por los propios dueños del negocio. Dicha aplicación funcionará como una vitrina digital que permitirá a cualquier visitante explorar el catálogo de productos disponibles, consultar descripciones, precios, tallas, descuentos y reseñas, sin necesidad de contactar directamente a los vendedores para obtener dicha información.

Por otro lado, la aplicación contará con un panel de administración que permitirá a los dueños y al personal designado gestionar el contenido del sitio de manera sencilla e intuitiva, incluyendo la publicación de productos, actualización de precios, gestión de promociones y administración de reseñas, con el objetivo de mantener el inventario siempre actualizado y visible para sus clientes.

Objetivo General y Objetivos Específicos

Objetivo General

Desarrollar una aplicación web de carácter publicitario denominada ShadowAngels, que permita a los visitantes explorar el inventario disponible del negocio de reventa de ropa de etiqueta de manera autónoma, visualizando fotografías, descripciones, precios, tallas, descuentos y reseñas de los productos, al mismo tiempo que proporcione a los administradores las herramientas necesarias para gestionar dicho contenido de forma sencilla e intuitiva, con el fin de fortalecer la presencia digital del negocio y optimizar su estrategia de publicidad.

Objetivos Específicos

1. Implementar un catálogo de productos accesible para cualquier visitante, sin necesidad de registro, que muestre fotografías, descripciones, precios, tallas, descuentos y reseñas de cada artículo disponible.
2. Desarrollar un sistema de registro opcional para usuarios que deseen recibir notificaciones sobre promociones y descuentos, así como la posibilidad de dejar reseñas en los productos de su interés.
3. Diseñar e implementar un panel de administración que permita a los administradores gestionar el inventario de productos mediante operaciones de creación, edición y eliminación, incluyendo la carga de imágenes y la configuración de precios y descuentos.
4. Establecer un sistema de notificaciones interno dentro de la plataforma, mediante el cual los administradores puedan informar a los usuarios registrados sobre nuevas promociones o descuentos disponibles, con confirmación previa antes de su publicación.
5. Implementar una jerarquía de roles de usuario que distinga entre usuario anónimo, usuario registrado, administrador general y super administrador, garantizando que cada perfil acceda únicamente a las funcionalidades que le corresponden.
6. Desarrollar un módulo de gestión de administradores accesible exclusivamente para los super administradores, que permita asignar, modificar y revocar roles de administrador general dentro del sistema.
7. Construir la aplicación web utilizando el stack tecnológico definido por la materia, compuesto por Angular y Bootstrap para el frontend, Python con Flask para el backend y MongoDB como sistema de gestión de base de datos.

Alcance

La presente sección describe de manera detallada las funcionalidades que serán y no serán contempladas dentro del desarrollo de la aplicación web ShadowAngels, definidas con base en las entrevistas realizadas con la clienta y ajustadas a los tiempos y recursos disponibles del equipo de desarrollo.

El sistema incluye:

- Catálogo de productos visible para cualquier visitante sin necesidad de registro, con soporte para múltiples fotografías por producto, descripción, precio, tallas disponibles, descuentos y reseñas.
- Sistema de registro e inicio de sesión opcional para usuarios, cuya finalidad es habilitar el acceso al apartado de notificaciones y la posibilidad de dejar reseñas en los productos.
- Apartado de notificaciones visible únicamente para usuarios registrados, donde se publican avisos de promociones y descuentos generados por los administradores. Los visitantes no registrados verán un mensaje invitándolos a crear una cuenta.
- Sistema de reseñas por producto, limitado a una reseña por usuario registrado por producto, gestionadas por los administradores con capacidad de eliminación en caso de mal uso.
- Panel de administración con inicio de sesión exclusivo, accesible únicamente para administradores generales y super administradores, desde el cual se gestiona todo el contenido del sitio.
- Módulo de gestión de productos con operaciones completas de creación, edición y eliminación, incluyendo carga de múltiples imágenes, descripción, precio, tallas y configuración de descuentos.
- Módulo de gestión de notificaciones desde el panel de administración, mediante el cual el administrador puede redactar y publicar un aviso dirigido a todos los usuarios registrados, con confirmación del sistema antes de su envío.
- Módulo de gestión de reseñas desde el panel de administración, con capacidad de eliminación de reseñas inapropiadas o que representen mal uso de la plataforma.
- Módulo de gestión de administradores accesible exclusivamente para super administradores, con operaciones de creación, edición y eliminación de cuentas de

administrador general, con la restricción de que un super administrador no puede eliminarse ni degradarse a sí mismo.

- Sección de contacto visible para todos los visitantes, con información de los medios de comunicación disponibles para contactar a los vendedores.
- Perfil de usuario editable tanto para usuarios registrados como para administradores.

El sistema no incluye:

- Pasarela de pagos en línea ni procesamiento de transacciones de ningún tipo, dado que el objetivo del sistema es exclusivamente publicitario.
- Módulo de gestión de ventas ni seguimiento de pedidos, ya que estas operaciones continuarán realizándose a través de los canales actuales del negocio.
- Módulo de gestión de entregas o logística.
- Notificaciones por correo electrónico o mensajería externa, dado que su implementación requiere servicios de terceros con costo que se encuentran fuera del alcance del proyecto.
- Aplicación móvil nativa, el sistema será desarrollado como aplicación web con diseño responsivo.
- Registro público de super administradores, ya que este rol es asignado manualmente por el desarrollador durante el despliegue inicial del sistema.

Restricciones y Supuestos

La presente sección establece las limitaciones técnicas, operativas y de contexto bajo las cuales se desarrollará el proyecto, así como los supuestos sobre los que se basa el diseño del sistema. Estas consideraciones son relevantes para delimitar el alcance del trabajo del equipo de desarrollo y evitar ambigüedades durante el proceso.

Restricciones tecnológicas

- El frontend de la aplicación deberá desarrollarse obligatoriamente con Angular y Bootstrap, conforme a los requisitos establecidos por la materia de Desarrollo de Aplicaciones Web.
- El backend deberá implementarse con Python utilizando el framework Flask, encargado de gestionar la lógica del negocio y la comunicación con la base de datos.
- El sistema de gestión de base de datos a utilizar será MongoDB, base de datos no relacional orientada a documentos.
- El desarrollo se limitará a una aplicación web con diseño responsivo, sin contemplar el desarrollo de una aplicación móvil nativa.

Restricciones operativas

- La aplicación tiene un propósito exclusivamente publicitario, por lo que no se implementarán funcionalidades de comercio electrónico, procesamiento de pagos ni gestión logística de ningún tipo.
- Las notificaciones a usuarios registrados serán internas a la plataforma, descartando el uso de servicios externos de correo electrónico o mensajería instantánea debido a que su implementación implica el uso de APIs de pago fuera del presupuesto del proyecto.
- La creación del primer Super Administrador del sistema se realizará directamente por el equipo de desarrollo mediante inserción manual en la base de datos durante el despliegue inicial, proceso que queda fuera del alcance visual de la aplicación y estará sujeto a los acuerdos de confidencialidad establecidos entre el equipo y la clienta.
- Un Super Administrador no podrá eliminarse a sí mismo ni modificar su propio rol dentro del sistema, con el fin de garantizar que siempre exista al menos un Super Administrador activo.

Restricciones de tiempo

- El desarrollo del proyecto está sujeto a los tiempos y entregas establecidos por el calendario escolar de la materia, por lo que el alcance definido en este documento representa la versión inicial del sistema y podrá estar sujeto a ajustes conforme avance el proceso de desarrollo.

Supuestos

- Se asume que la clienta proporcionará oportunamente el material necesario para poblar el catálogo inicial de productos, incluyendo fotografías, descripciones y precios.
- Se asume que los Super Administradores, correspondientes a los dueños del negocio, contarán con acceso a un dispositivo con conexión a internet para gestionar el contenido de la plataforma.
- Se asume que los Super Administradores serán capacitados por el equipo de desarrollo para el uso del panel de administración antes de la entrega final del proyecto.
- Se asume que el contenido publicado en la plataforma, incluyendo fotografías y descripciones de productos, es propiedad de la clienta o cuenta con los permisos necesarios para su uso, por lo que la responsabilidad sobre dicho contenido recae en los administradores del sistema.
- Se asume que la aplicación será desplegada en un entorno con acceso a internet y que el equipo de desarrollo será responsable de la configuración inicial del servidor durante el periodo escolar.

Usuarios del Sistema

La presente sección describe los cuatro perfiles de usuario identificados dentro del sistema ShadowAngels, detallando las funcionalidades correspondientes a cada uno. La distinción entre perfiles es fundamental para garantizar que cada usuario acceda únicamente a las funciones que le corresponden según su rol dentro de la plataforma.

1. Usuario Anónimo

Es cualquier visitante que accede a la aplicación web sin haber creado una cuenta ni iniciado sesión. Este perfil no requiere ningún tipo de autenticación y representa al público general al que va dirigida la plataforma como herramienta publicitaria.

Funcionalidades disponibles:

- Explorar el catálogo de productos organizado por categorías.
- Visualizar fotografías de los productos.
- Leer descripciones, precios, tallas disponibles y descuentos de cada producto.
- Leer reseñas dejadas por otros usuarios registrados.
- Consultar la sección de contacto para comunicarse con los vendedores.
- Visualizar un mensaje de invitación al registro en el apartado de notificaciones.

Funcionalidades no disponibles:

- Dejar reseñas en productos.
- Acceder al apartado de notificaciones.
- Acceder al panel de administración.

2. Usuario Registrado

Es un visitante que ha creado una cuenta de manera voluntaria dentro de la plataforma. El registro no es obligatorio para explorar el catálogo, sin embargo, habilita funcionalidades adicionales orientadas a mejorar la experiencia del usuario y mantenerlo informado sobre promociones y descuentos disponibles.

Funcionalidades disponibles:

- Todas las funcionalidades del usuario anónimo.
- Acceder al apartado de notificaciones para consultar avisos de promociones y descuentos publicados por los administradores.
- Eliminar notificaciones individuales desde su apartado personal.
- Dejar una reseña por producto, con calificación y comentario.
- Gestionar su perfil de usuario.

Funcionalidades no disponibles:

- Editar o eliminar reseñas de otros usuarios.
- Acceder al panel de administración.

3. Administrador General

Es un usuario con acceso al panel de administración, designado por un Super Administrador desde el módulo de gestión de administradores. Este perfil no puede registrarse de manera pública, su cuenta es creada exclusivamente desde el panel de administración por un Super Administrador.

Funcionalidades disponibles:

- Iniciar sesión en el panel de administración.
- Gestionar productos mediante operaciones de creación, edición y eliminación.
- Cargar y administrar múltiples fotografías por producto.
- Configurar y actualizar precios, tallas y descuentos de los productos.
- Agregar y editar descripciones de productos.
- Eliminar reseñas de usuarios en caso de mal uso o contenido inapropiado.
- Redactar y publicar notificaciones dirigidas a todos los usuarios registrados, con confirmación del sistema previa a su envío.
- Actualizar la información de contacto visible en la plataforma.
- Gestionar su perfil de administrador.

Funcionalidades no disponibles:

- Gestionar cuentas de otros administradores.

- Crear, editar o eliminar cuentas de Super Administrador.
- Acceder al módulo de gestión de administradores.

4. Super Administrador

Es el rol de mayor jerarquía dentro del sistema, reservado para los dueños del negocio. Las cuentas con este rol son creadas manualmente por el equipo de desarrollo durante el despliegue inicial del sistema, conforme a los acuerdos de confidencialidad establecidos con la clienta. Este perfil concentra el control total sobre la plataforma.

Funcionalidades disponibles:

- Todas las funcionalidades del Administrador General.
- Acceder al módulo de gestión de administradores.
- Crear, editar y eliminar cuentas de Administrador General.
- Asignar y revocar el rol de Administrador General a los usuarios designados.

Restricciones del sistema aplicadas a este rol:

- Un Super Administrador no puede eliminar su propia cuenta.
- Un Super Administrador no puede modificar ni revocar su propio rol.
- Estas restricciones garantizan que el sistema cuente siempre con al menos un Super Administrador activo.

Historias de Usuario

La presente sección describe las funcionalidades esperadas del sistema desde la perspectiva de cada perfil de usuario, redactadas en formato de historia de usuario. Cada historia incluye el rol, la acción deseada, el objetivo que la motiva y la condición mínima de aceptación que define cuándo se considera correctamente implementada.

Usuario Anónimo

HU-01 — Explorar el catálogo de productos *Como* usuario anónimo, *quiero* poder explorar el catálogo de productos disponibles organizados por categoría, *para* conocer la oferta del negocio sin necesidad de registrarme. *Condición de aceptación:* El catálogo es visible para cualquier visitante sin requerir inicio de sesión, y los productos se muestran agrupados por categoría con su fotografía, nombre y precio.

HU-02 — Ver detalle de un producto *Como* usuario anónimo, *quiero* poder acceder al detalle de un producto, *para* consultar sus fotografías, descripción completa, precio, tallas disponibles, descuentos y reseñas de otros usuarios. *Condición de aceptación:* Al seleccionar un producto, se despliega una vista con toda la información del mismo, incluyendo múltiples fotografías si las hubiera, y las reseñas asociadas al producto si existieran.

HU-03 — Consultar información de contacto *Como* usuario anónimo, *quiero* poder consultar los medios de contacto disponibles del negocio, *para* comunicarme directamente con los vendedores en caso de estar interesado en algún producto. *Condición de aceptación:* La sección de contacto es visible para cualquier visitante y muestra la información de contacto actualizada por los administradores.

HU-04 — Como usuario no registrado, quiero ver un banner en la pantalla de inicio que me muestre los beneficios de registrarme, para conocer las ventajas de crear una cuenta. **Condición de aceptación:** Al acceder a la pantalla de inicio sin haber iniciado sesión, el sistema muestra un banner visible con información sobre los beneficios de registrarse, incentivando al usuario a crear una cuenta.

Usuario Registrado

HU-05 — Registrarse en la plataforma *Como* visitante, *quiero* poder crear una cuenta de manera voluntaria, *para* acceder a funcionalidades adicionales como notificaciones y reseñas. *Condición*

de aceptación: El sistema permite crear una cuenta con los datos básicos requeridos. El registro es completamente opcional y no restringe el acceso al catálogo ni a la información de productos.

HU-06 — Iniciar y cerrar sesión *Como* usuario registrado, *quiero* poder iniciar y cerrar sesión en mi cuenta, *para* acceder a mis funcionalidades personales de manera segura. *Condición de aceptación:* El sistema permite iniciar sesión con las credenciales registradas y cerrar sesión en cualquier momento, redirigiendo al usuario a la vista de usuario anónimo.

HU-07 — Recibir y consultar notificaciones *Como* usuario registrado, *quiero* poder acceder al apartado de notificaciones, *para* estar informado sobre promociones y descuentos publicados por los administradores. *Condición de aceptación:* El apartado de notificaciones muestra los avisos publicados por los administradores en orden cronológico, con fecha de publicación visible.

HU-08 — Eliminar notificaciones *Como* usuario registrado, *quiero* poder eliminar notificaciones de mi apartado personal, *para* mantener organizada mi bandeja de notificaciones. *Condición de aceptación:* El sistema permite al usuario eliminar notificaciones individuales desde su apartado, sin afectar las notificaciones de otros usuarios.

HU-09 — Dejar una reseña en un producto *Como* usuario registrado, *quiero* poder dejar una reseña con calificación y comentario en un producto, *para* compartir mi experiencia con otros usuarios interesados. *Condición de aceptación:* El sistema permite al usuario registrado dejar únicamente una reseña por producto. Si el usuario ya dejó una reseña en ese producto, la opción de agregar reseña no estará disponible para ese producto.

HU-10 — Gestionar mi perfil *Como* usuario registrado, *quiero* poder editar la información de mi perfil, *para* mantener mis datos actualizados dentro de la plataforma. *Condición de aceptación:* El sistema permite al usuario editar su información de perfil y guardar los cambios de manera exitosa.

Administrador General

HU-11 — Iniciar sesión en el panel de administración *Como* administrador general, *quiero* poder iniciar sesión en el panel de administración con mis credenciales, *para* acceder a las herramientas de gestión del contenido de la plataforma. *Condición de aceptación:* El sistema valida las credenciales del administrador y redirige al panel de administración. El acceso es denegado si las credenciales son incorrectas.

HU-12 — Gestionar productos *Como* administrador general, *quiero* poder crear, editar y eliminar productos del catálogo, *para* mantener el inventario de la plataforma actualizado. *Condición de aceptación:* El sistema permite crear un producto nuevo con todos sus atributos, editar cualquier

campo de un producto existente y eliminar productos del catálogo. Los cambios se reflejan de manera inmediata en la vista del usuario.

HU-13 — Cargar fotografías de productos *Como* administrador general, *quiero* poder cargar múltiples fotografías por producto, *para* que los clientes puedan apreciar los productos con detalle desde distintos ángulos. *Condición de aceptación:* El sistema permite cargar y asociar múltiples imágenes a un mismo producto durante su creación o edición.

HU-14 — Gestionar precios y descuentos *Como* administrador general, *quiero* poder configurar y actualizar el precio y los descuentos de cada producto, *para* mantener la información comercial siempre vigente y llamar la atención de los clientes con promociones. *Condición de aceptación:* El sistema permite establecer y modificar el precio regular y el descuento aplicable de cada producto. El precio con descuento se calcula y muestra de manera automática en la vista del usuario.

HU-15 — Publicar notificaciones a usuarios registrados *Como* administrador general, *quiero* poder redactar y publicar una notificación dirigida a todos los usuarios registrados, *para* informarles sobre nuevas promociones o descuentos disponibles en la plataforma. *Condición de aceptación:* El sistema permite al administrador redactar el mensaje de la notificación y confirmar su publicación. Tras la confirmación, la notificación aparece en el apartado de notificaciones de todos los usuarios registrados.

HU-16 — Eliminar reseñas inapropiadas *Como* administrador general, *quiero* poder eliminar reseñas de usuarios que representen mal uso de la plataforma, *para* garantizar que el contenido publicado sea apropiado y útil para los visitantes. *Condición de aceptación:* El sistema permite al administrador eliminar cualquier reseña desde el panel de administración. La eliminación es permanente y se refleja de inmediato en la vista del producto correspondiente.

HU-17 — Actualizar información de contacto *Como* administrador general, *quiero* poder actualizar los medios de contacto visibles en la plataforma, *para* asegurarme de que los clientes siempre tengan acceso a información de contacto vigente. *Condición de aceptación:* El sistema permite editar y guardar la información de contacto desde el panel de administración, reflejándose de inmediato en la sección de contacto visible para todos los visitantes.

Super Administrador

HU-18 — Crear cuenta de Administrador General *Como* super administrador, *quiero* poder crear cuentas de administrador general desde el panel de gestión de administradores, *para* asignar acceso al panel de administración a las personas que yo designe. *Condición de aceptación:* El sistema

permite al super administrador crear una nueva cuenta de administrador general con los datos requeridos, la cual queda activa de manera inmediata.

HU-19 — Editar cuenta de Administrador General *Como* super administrador, *quiero* poder editar la información de las cuentas de administrador general existentes, *para* mantener los datos de mis administradores actualizados. *Condición de aceptación:* El sistema permite modificar los datos de cualquier cuenta de administrador general y guardar los cambios de manera exitosa.

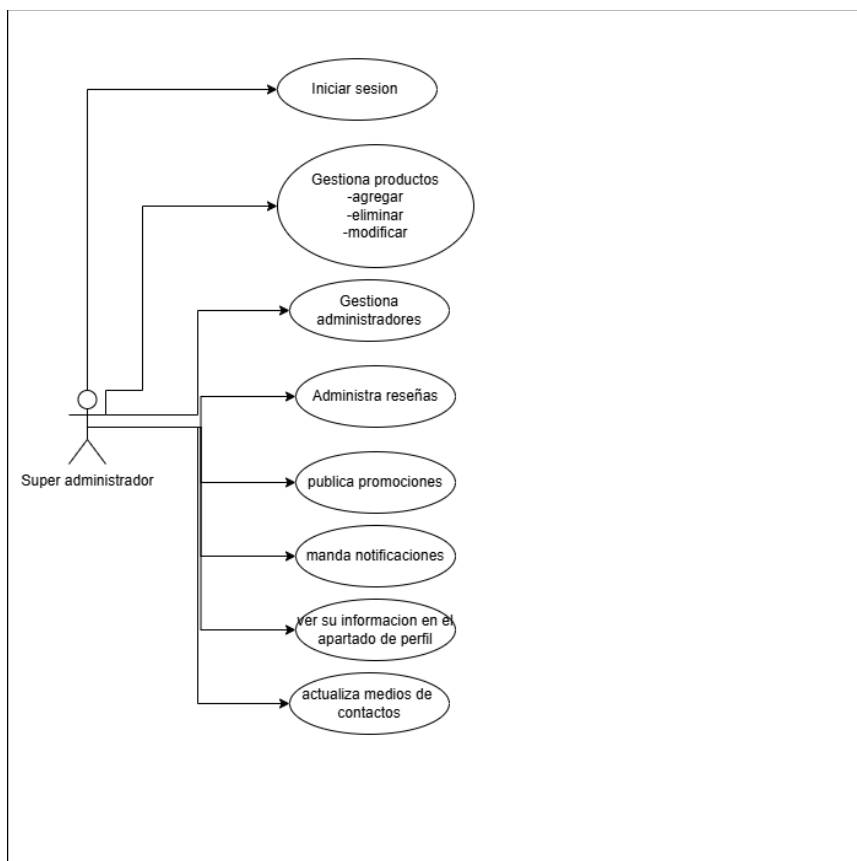
HU-20 — Eliminar cuenta de Administrador General *Como* super administrador, *quiero* poder eliminar cuentas de administrador general cuando sea necesario, *para* revocar el acceso al panel de administración a personas que ya no deban tenerlo. *Condición de aceptación:* El sistema permite eliminar cuentas de administrador general de manera permanente. El sistema no permite al super administrador eliminar su propia cuenta ni modificar su propio rol.

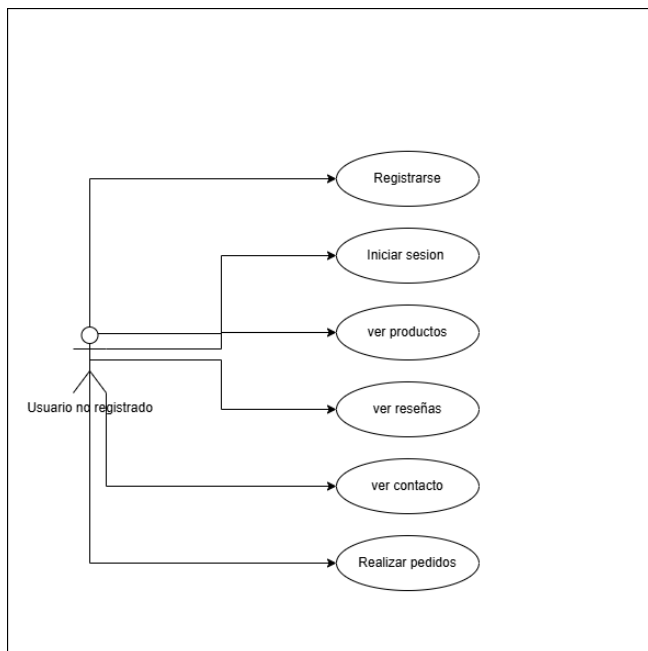
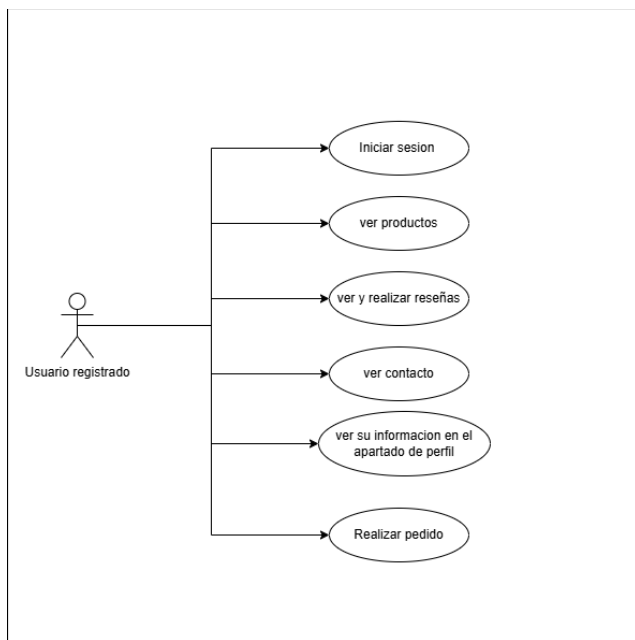
Diagramas

La presente sección contiene las representaciones gráficas del sistema ShadowAngels, organizadas en dos tipos de diagramas: el diagrama de casos de uso y el diagrama de navegación. Ambos fueron elaborados con base en la información recopilada durante las entrevistas con la clienta y en los acuerdos establecidos por el equipo de desarrollo, y tienen como propósito ilustrar de manera clara las interacciones esperadas entre los usuarios y el sistema, así como el flujo de navegación previsto para cada perfil.

Diagrama de Casos de Uso

El diagrama de casos de uso representa las interacciones funcionales entre los distintos perfiles de usuario y el sistema. Se identifican cuatro actores principales: el Usuario Anónimo, el Usuario Registrado, el Administrador General y el Super Administrador, cada uno con acceso a los casos de uso correspondientes a su rol según lo definido en la sección de Usuarios del Sistema.





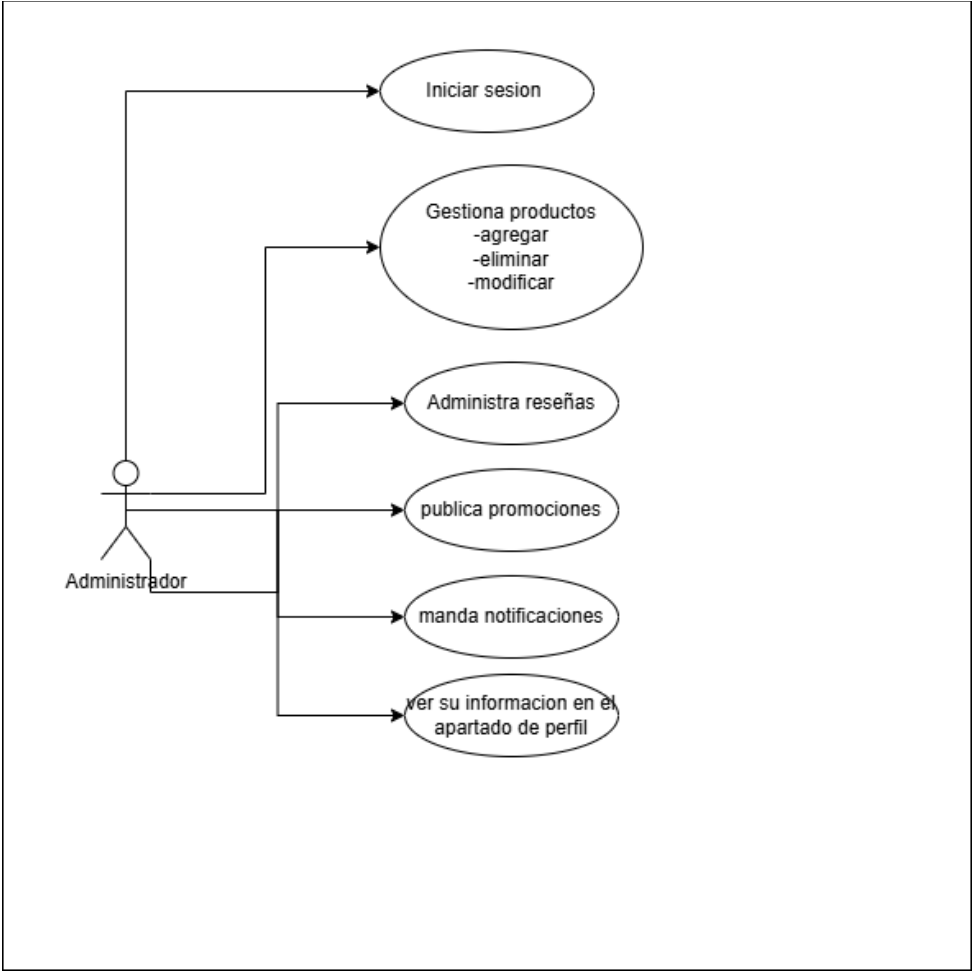
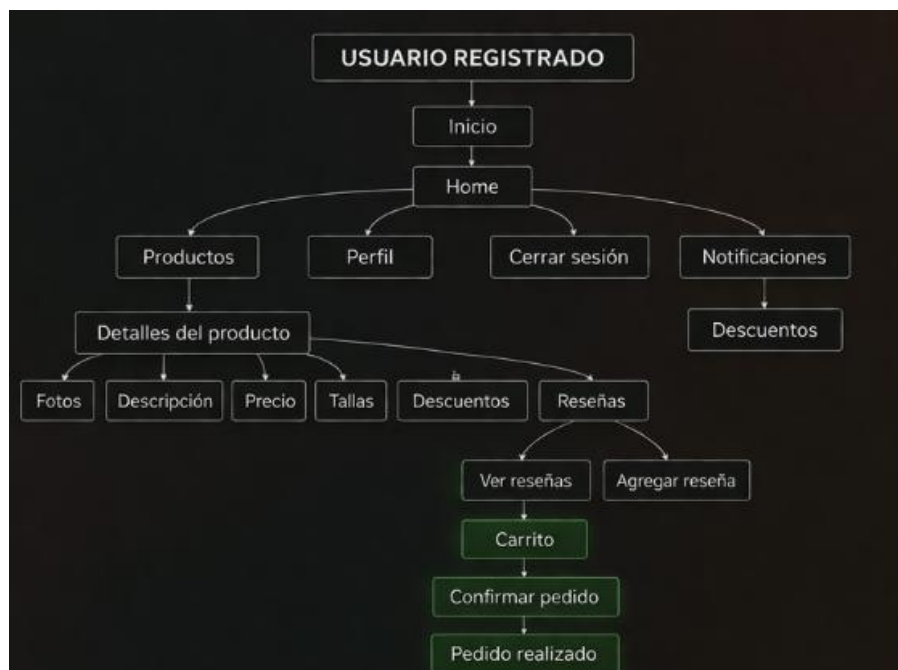
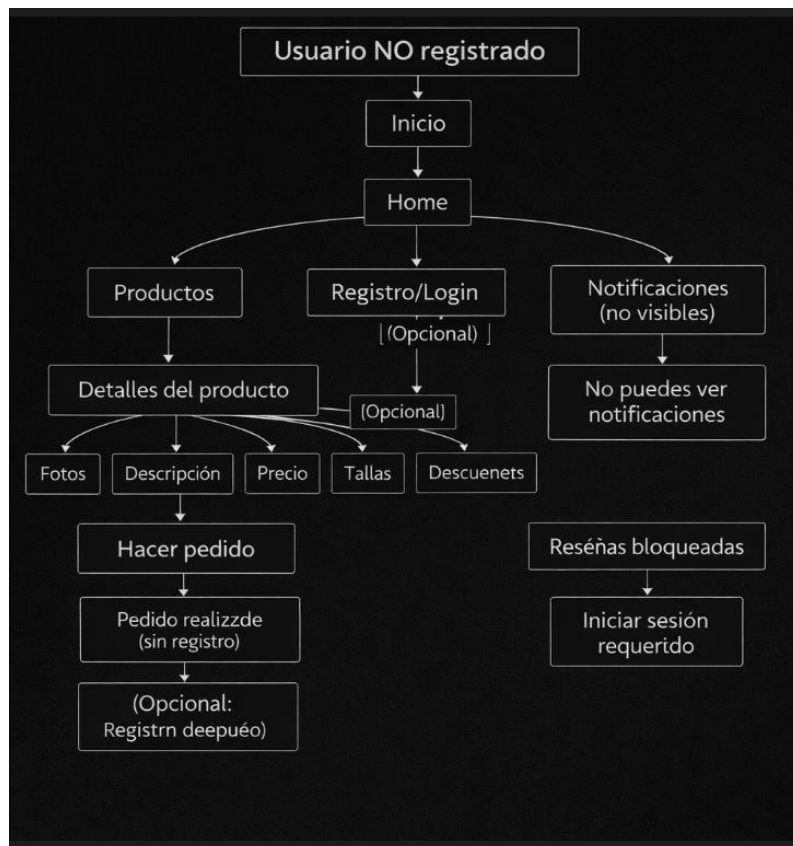
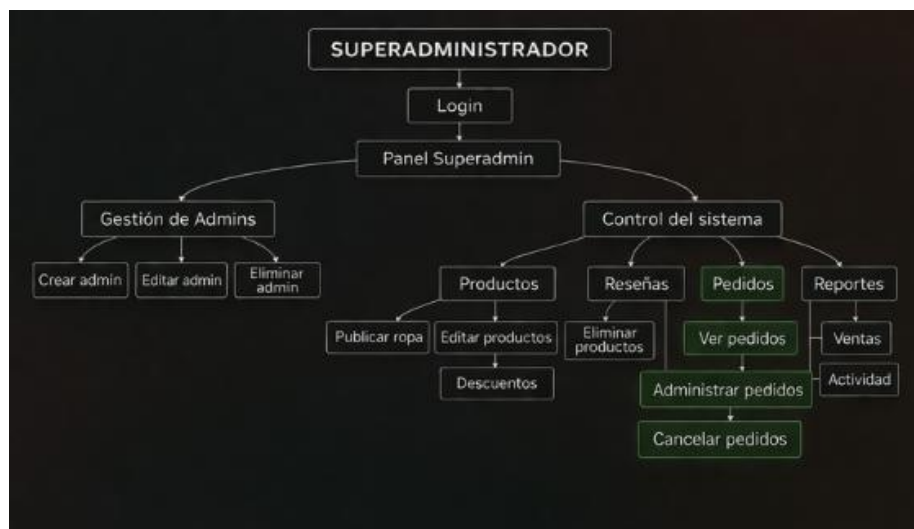
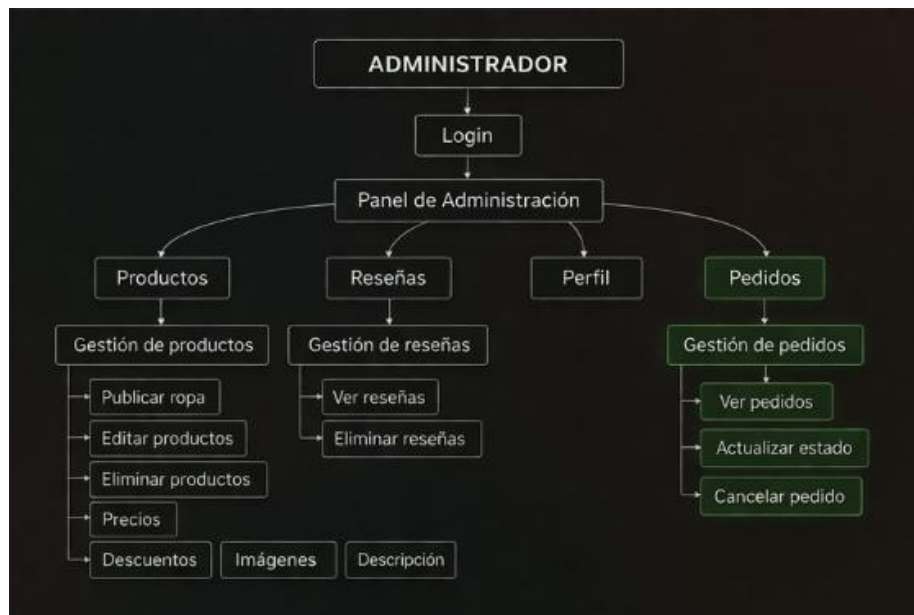


Diagrama de Navegación

El diagrama de navegación ilustra el flujo de pantallas y rutas disponibles para cada perfil de usuario dentro de la aplicación. Se presentan dos flujos diferenciados: el flujo de navegación del usuario, que parte desde la página de inicio y da acceso al catálogo, detalle de productos, perfil, notificaciones y registro, y el flujo de navegación del administrador, que parte desde el login de administración y da acceso al panel de administración con sus respectivos módulos de gestión.





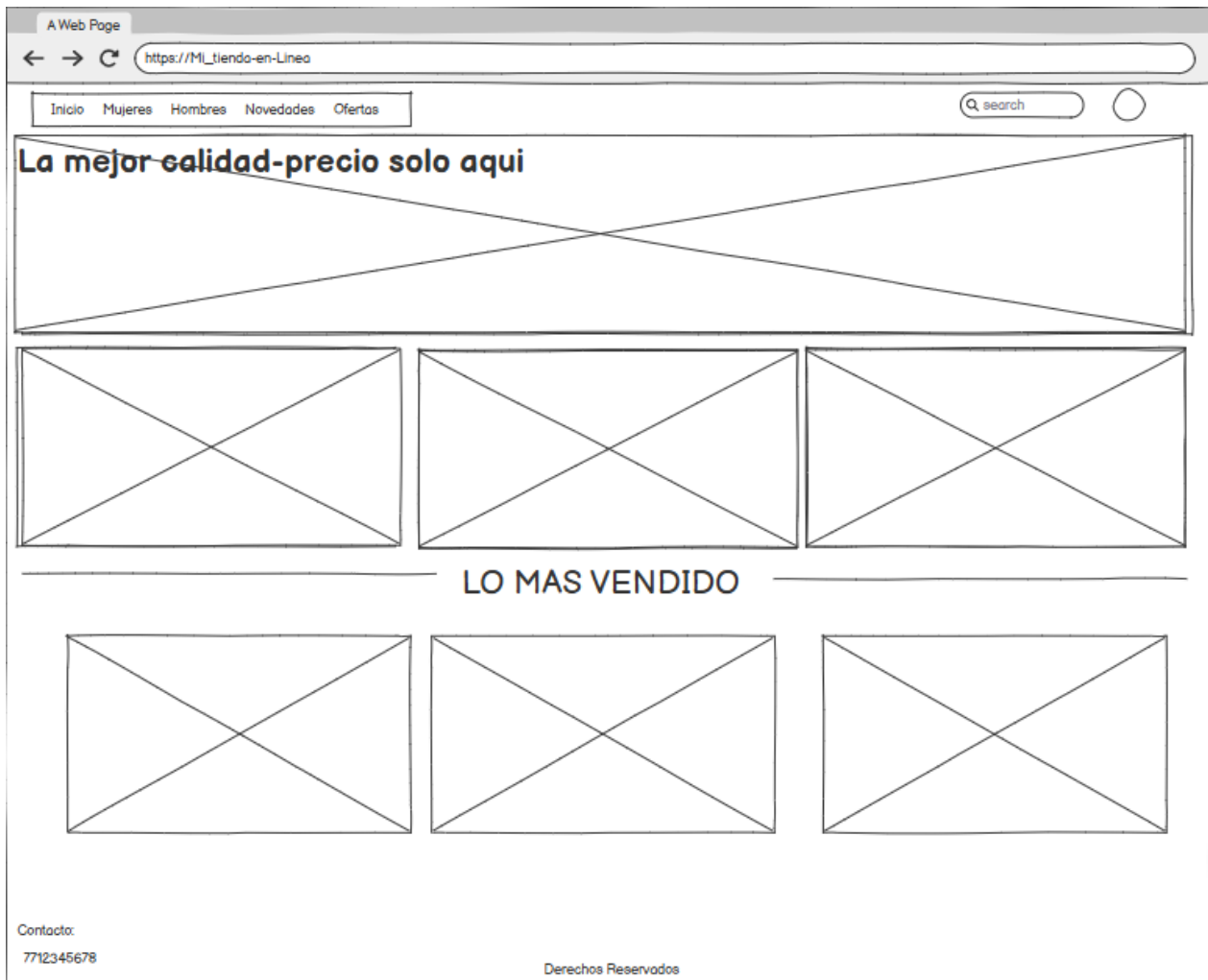
Wireframes

La presente sección contiene los wireframes iniciales de la aplicación web ShadowAngels, los cuales representan la estructura visual preliminar de cada pantalla del sistema. Estos diseños fueron elaborados como propuesta inicial y podrán estar sujetos a modificaciones conforme avance el proceso de desarrollo y se reciba retroalimentación de la cliente.

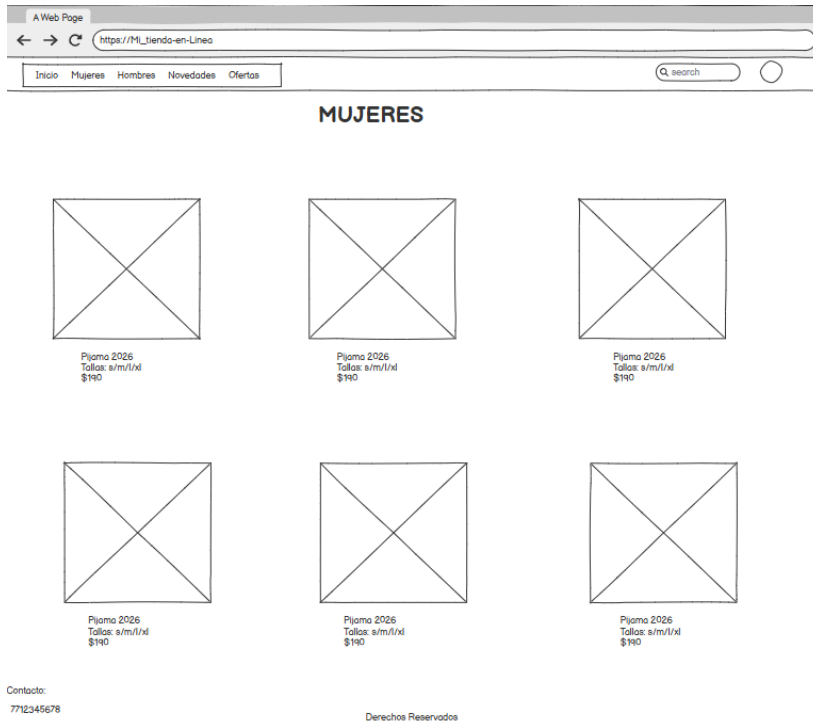
Los wireframes se organizan en dos grupos: las pantallas correspondientes a la vista del usuario y las pantallas correspondientes al panel de administración.

Pantallas de Usuario

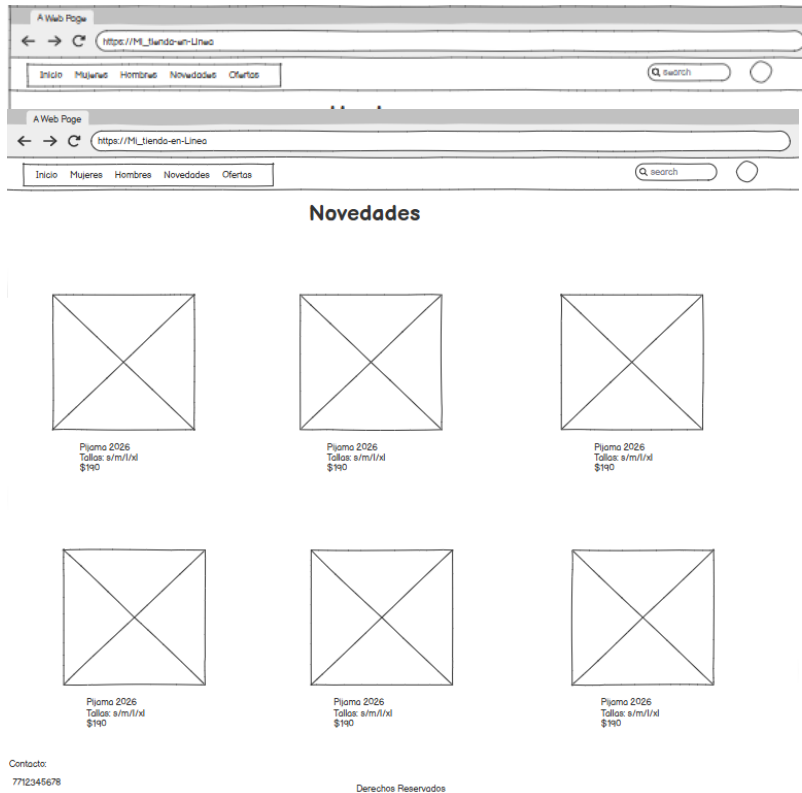
Inicio — Página principal de la aplicación, presenta un banner principal con el slogan del negocio, una sección de categorías destacadas y una sección de productos más vendidos. Incluye barra de navegación con acceso a las categorías Mujeres, Hombres, Novedades y Ofertas, buscador y acceso al perfil o registro.



Mujeres — Vista del catálogo filtrado por la categoría Mujeres, presenta los productos disponibles en formato de cuadrícula con fotografía, nombre, tallas disponibles y precio.

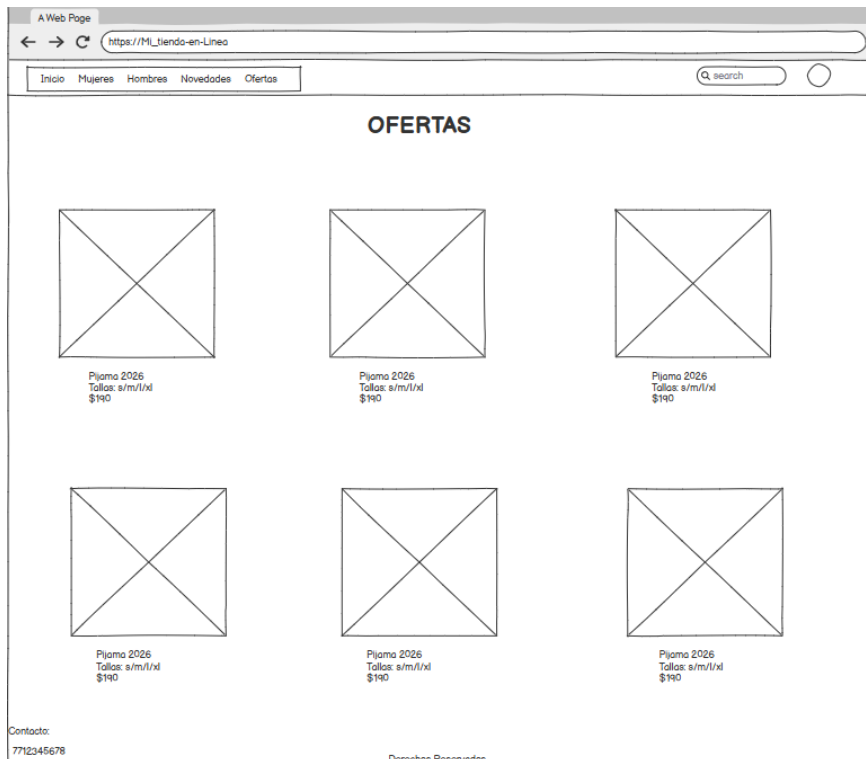


Hombres — Vista del catálogo filtrado por la categoría Hombres, con la misma estructura de cuadrícula que la vista de Mujeres.



Novedades — Vista del catálogo filtrado por productos de reciente incorporación al inventario, con la misma estructura de cuadrícula.

Ofertas — Vista del catálogo filtrado por productos con descuento activo, con la misma estructura de cuadrícula.

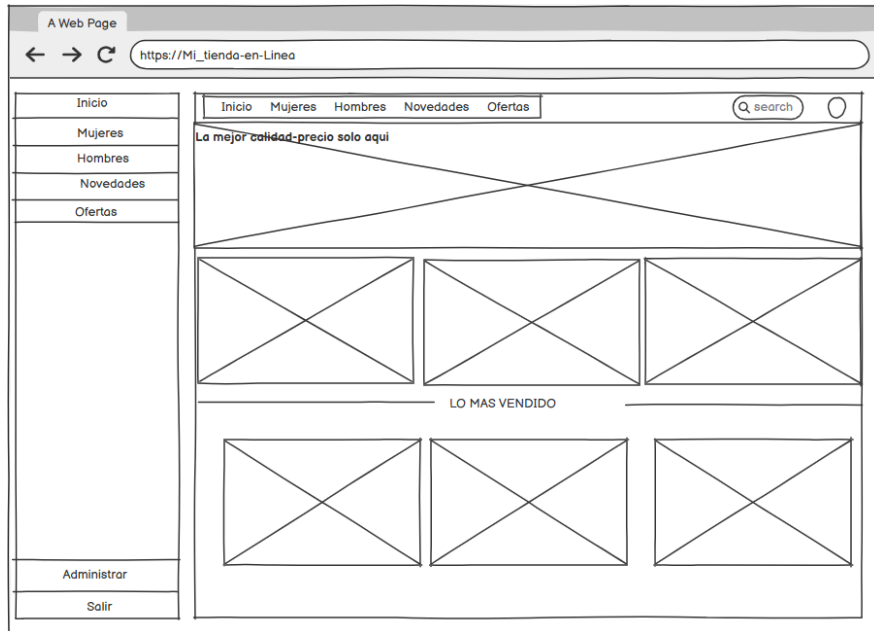


Pantallas del Panel de Admin

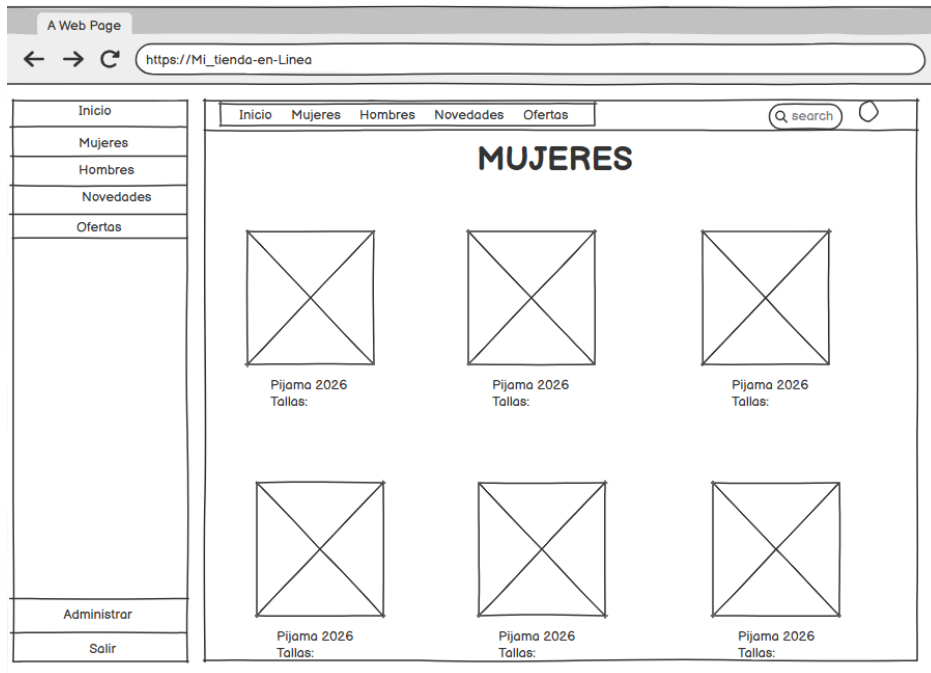
Login de Administración — Pantalla de autenticación exclusiva para administradores generales y super administradores. Presenta campos de usuario y contraseña, botón de acceso y enlace para restablecimiento de contraseña.



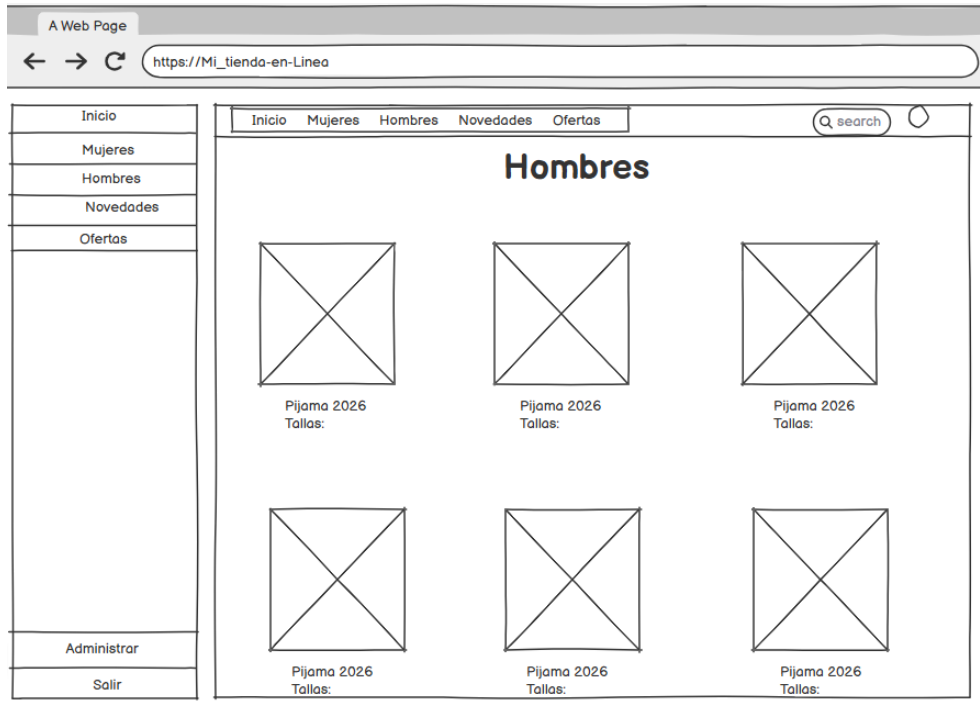
Inicio Admin — Panel principal del administrador, presenta la misma vista del catálogo que el usuario con la adición de una barra lateral de navegación con acceso a los módulos de gestión y opciones de administrar y cerrar sesión.



Mujeres Admin — Vista de la categoría Mujeres desde el panel de administración, con la barra lateral de navegación y acceso a las opciones de gestión de productos de esta categoría.



Hombres Admin — Vista de la categoría Hombres desde el panel de administración, con la misma estructura que Mujeres Admin.



Novedades Admin — Vista de la sección Novedades desde el panel de administración, con la misma estructura que las categorías anteriores.

Pantallas del panel SuperAdmin

Inicio de SuperAdmin — Pantalla de que muestra los productos que hay agregados, si tienen descuento, son nuevos y muestra la lista de los productos

The screenshot shows a web browser window with the URL `https://Mi_tienda-en-Linea`. The page has a sidebar menu on the left with options: Inicio, Productos, Solicitudes, Notificaciones, Reseñas, Perfil, Administradores, Contacto, Personalización, and Salir. The main content area has a top navigation bar with links: Inicio, Mujeres, Hombres, Novedades, Ofertas, and Panel. Below the navigation bar, the page is titled "Panel de Administración" and "Bienvenido,raul (superadmin)". The main content area displays three summary boxes: "Total de Productos", "Con descuento", and "Productos nuevos". Below these boxes, there is a list of products, with the first item being "Shampoo Bonito y elegante" priced at "\$140".

Productos de SuperAdmin — Pantalla que muestra el apartado para crear nuevos productos

The screenshot shows the "Gestión de Productos" section of the SuperAdmin dashboard. The page title is "Gestión de Productos" and the sub-header is "Crear producto". The form includes the following fields: "Nombre" (text input), "Categoría" (dropdown menu with "Dama" selected), "Descripción" (text area), "Precio" (text input), "Descuento" (text input), "Tallas" (text input with "XS,S,M,L,XL" entered), and a checkbox labeled "Marcar como nuevo". A "Guardar" button is located at the bottom of the form. The sidebar and top navigation bar are identical to the previous screenshot.

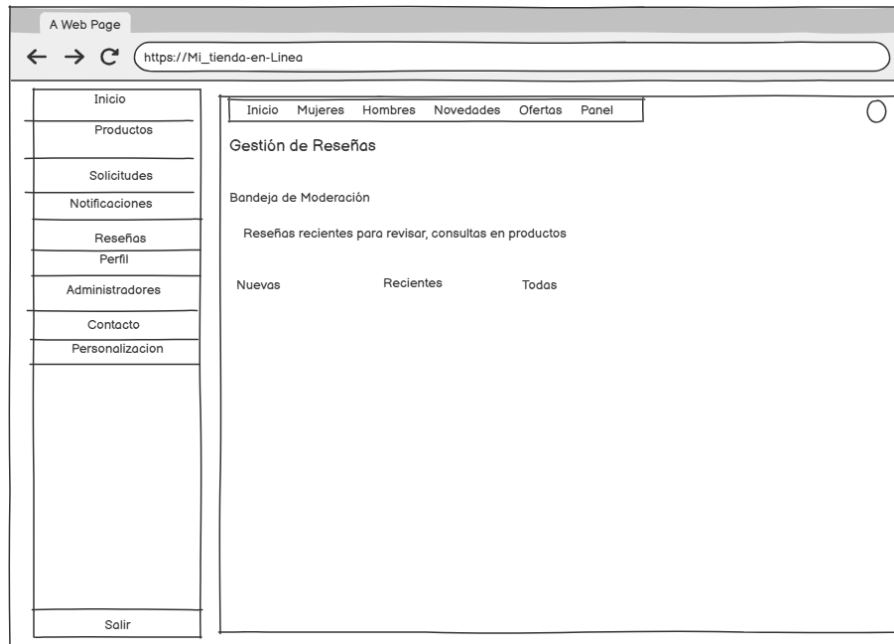
Solicitudes de SuperAdmin — Pantalla de que muestra las solicitudes al realizar una compra

This wireframe represents the 'Solicitudes de SuperAdmin' page. It features a browser window at the top with the address 'https://Mi_tienda-en-Linea'. On the left is a vertical sidebar menu with the following items: Inicio, Productos, Solicitudes (highlighted), Notificaciones, Reseñas, Perfil, Administradores, Contacto, Personalizacion, and Salir. The main content area has a top navigation bar with links: Inicio, Mujeres, Hombres, Novedades, Ofertas, and Panel. Below this, the section is titled 'Gestión de Solicitudes'. It contains a row of filter tabs: Todas, pendientes, Confirmados, No realizadas, and Recientes. The area below the tabs is a large empty rectangle, intended for a table of requests.

Notificaciones de SuperAdmin — Pantalla de que permite crear notificaciones, para los usuarios que estén registrados

This wireframe represents the 'Notificaciones de SuperAdmin' page. It includes the same browser window and sidebar menu as the previous page. The main content area has a top navigation bar with links: Inicio, Mujeres, Hombres, Novedades, Ofertas, and Panel. The section is titled 'Gestión de Notificaciones'. It is divided into two main panels. The left panel, titled 'Nueva Notificación', contains three text input fields labeled 'Título', 'Resumen', and 'Contenido Completo', followed by a 'Guardar' button. The right panel, titled 'Notificaciones creadas', is a large empty rectangle for displaying a list of created notifications.

Reseñas de SuperAdmin — Pantalla de que muestra todas las reseñas que se dejan en los productos que han comprado los clientes registrados



Perfil de SuperAdmin — Pantalla de que muestra los datos que tienen los super administradores, se pueden modificar los datos, como el nombre, correo electrónico

A Web Page

← → ↻ https://Mi_tienda-en-Linea

Inicio Mujeres Hombres Novedades Ofertas Panel

Gestión de Perfil SuperAdmin

Cuenta

Mi perfil
revisa tu información personal y actualiza los datos

Nombre Completo

Correo Electrónico

Rol Estado de cuenta

Recargar Guardar

Salir

Gestión de administradores de SuperAdmin — Pantalla de que muestra los administradores que han sido dados de alta para editar productos, reseñas, entre otras actividades

A Web Page

← → ↻ https://Mi_tienda-en-Linea

Inicio Mujeres Hombres Novedades Ofertas Panel

Gestión de Administradores

Crear Administrador

Nombre Correo Electrónico

Contraseña

Crear Admin

Lista de Administradores

Nombre	Correo	Rol	Acciones
--------	--------	-----	----------

Guardar

Salir

Contacto de SuperAdmin — Pantalla de que muestra las diferentes formas en que los clientes pueden contactar al dueño.

A Web Page

←

→

↻

https://Mi_tienda-en-Linea

Inicio

Productos

Solicitudes

Notificaciones

Reseñas

Perfil

Administradores

Contacto

Personalizació

Salir

Inicio

Mujeres

Hombres

Novedades

Ofertas

Panel

Contacto

Número de Whatsapp

Instagram

Tiktok

Descripción Ubicación

Texto visible Whatsapp

Facebook

Email

Guardar

Personalización de SuperAdmin — Pantalla que permite modificar el fondo de imagen del Inicio de la pagina

A Web Page

https://Mi_tienda-en-Linea

Inicio Mujeres Hombres Novedades Ofertas Panel

Inicio
Productos
Solicitudes
Notificaciones
Reseñas
Perfil
Administradores
Contacto
Personalización
Salir

Personalización

Título

Subtítulo

Imagen de fondo

Elegir Seleccionar Archivo

Quitar fondo

Guardar

Vista Previa

Inicio

Bienvenidos a ShadowAngels

Descubre nuestros Productos, novedades y ofertas

Estructura Inicial del Frontend

La presente sección describe la organización preliminar de los módulos y componentes que conformarán el frontend de la aplicación web ShadowAngels, desarrollado con Angular y Bootstrap. Esta estructura representa una propuesta inicial sujeta a modificaciones conforme avance el proceso de desarrollo.

La arquitectura del frontend se organiza en tres grandes módulos: el módulo de usuario, el módulo de administración y el módulo de componentes compartidos.

Módulo de Usuario /users

Contiene todas las vistas y componentes accesibles para usuarios anónimos y usuarios registrados. Es el módulo principal de la aplicación desde la perspectiva del visitante.

- Home — Página principal de la aplicación. Muestra el banner principal, las categorías destacadas y la sección de productos más vendidos.
- Dama — Vista del catálogo filtrado por la categoría Mujeres. Presenta los productos disponibles en formato de cuadrícula.
- Caballero — Vista del catálogo filtrado por la categoría Hombres, con la misma estructura que la vista Dama.
- Ofertas — Vista del catálogo filtrado por productos con descuento activo.
- Producto — Vista de detalle de un producto individual. Muestra fotografías, descripción completa, precio, tallas, descuentos y la sección de reseñas asociadas al producto.
- Perfil — Vista de gestión del perfil del usuario registrado. Permite editar información personal.
- Notificaciones — Apartado exclusivo para usuarios registrados donde se visualizan los avisos de promociones y descuentos publicados por los administradores. Los usuarios no registrados visualizarán en su lugar un mensaje de invitación al registro.

Módulo de Administración /admins

Contiene todas las vistas y componentes del panel de administración, accesibles únicamente para administradores generales y super administradores mediante autenticación previa.

- Home — Panel principal del administrador. Presenta la vista general del catálogo con acceso a los módulos de gestión desde la barra lateral de navegación.
- Productos — Módulo de gestión del catálogo de productos. Permite realizar operaciones de creación, edición y eliminación de productos, incluyendo carga de imágenes, configuración de precios, tallas y descuentos.
- Admins — Módulo de gestión de administradores generales, accesible exclusivamente para super administradores. Permite crear, editar y eliminar cuentas de administrador general, con la restricción de que un super administrador no puede eliminar ni modificar su propio rol.
- Perfil — Vista de gestión del perfil del administrador. Permite editar la información personal de la cuenta de administración.

Módulo Compartido /shared

Contiene los componentes reutilizables que son utilizados tanto en el módulo de usuario como en el módulo de administración, con el objetivo de mantener consistencia visual y evitar duplicidad de código.

- Navbar — Barra de navegación principal. Se adapta según el perfil del usuario activo, mostrando las opciones correspondientes a cada rol.
- Footer — Pie de página de la aplicación. Incluye la información de contacto del negocio y los derechos reservados.

Capturas Código

1. Services (admin.services.ts)

```
import { Injectable, inject } from '@angular/core';

import { HttpClient } from '@angular/common/http';
import { environment } from '../../environments/environment';
import { Review } from '../../shared/interfaces/review.interface';

@Injectable({
  providedIn: 'root'
})
export class AdminService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;

  getAdmins() {
    return this.http.get(`${this.apiUrl}/admins`);
  }

  createAdmin(data: any) {
    return this.http.post(`${this.apiUrl}/admins`, data);
  }

  updateAdmin(id: string, data: any) {
    return this.http.put(`${this.apiUrl}/admins/${id}`, data);
  }

  deleteAdmin(id: string) {
    return this.http.delete(`${this.apiUrl}/admins/${id}`);
  }

  createNotification(data: any) {
    return this.http.post(`${this.apiUrl}/notifications`, data);
  }

  getAdminReviews() {
    return this.http.get<Review[]>(`${this.apiUrl}/admin/reviews`);
  }

  deleteReview(id: string) {
    return this.http.delete<{ message: string }>(`${this.apiUrl}/reviews/${id}`);
  }
}
```



```
}  
}
```

2. Auth.services.ts

```
import { Injectable, inject } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
import { BehaviorSubject, Observable, tap } from 'rxjs';  
import { AuthResponse } from '../../../shared/interfaces/auth-response.interface';  
import { environment } from '../../../environments/environment';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class AuthService {  
  private http = inject(HttpClient);  
  private apiUrl = environment.apiUrl;  
  private authStateSubject = new BehaviorSubject<any |  
null>(this.readUserFromStorage());  
  
  readonly authState$ = this.authStateSubject.asObservable();  
  
  login(email: string, password: string): Observable<AuthResponse> {  
    return this.http.post<AuthResponse>(`${this.apiUrl}/auth/login`, { email,  
password }).pipe(  
      tap((response) => {  
        if (typeof window !== 'undefined' && typeof localStorage !== 'undefined')  
{  
          localStorage.setItem('token', response.token);  
          localStorage.setItem('user', JSON.stringify(response.user));  
        }  
        this.authStateSubject.next(response.user);  
      })  
    );  
  }  
  
  register(name: string, email: string, password: string): Observable<any> {  
    return this.http.post(`${this.apiUrl}/auth/register`, { name, email, password  
});  
  }  
  
  adminLogin(email: string, password: string) {
```

```

    return this.http.post<AuthResponse>(`${this.apiUrl}/admin/auth/login`, {
email, password }).pipe(
    tap((response) => {
        if (typeof window !== 'undefined' && typeof localStorage !== 'undefined')
    {
        localStorage.setItem('token', response.token);
        localStorage.setItem('user', JSON.stringify(response.user));
    }
    this.authStateSubject.next(response.user);
    })
    );
}

getToken(): string | null {
    if (typeof window === 'undefined' || typeof localStorage === 'undefined') {
        return null;
    }
    return localStorage.getItem('token');
}

getUser(): any | null {
    if (typeof window === 'undefined' || typeof localStorage === 'undefined') {
        return null;
    }

    const user = localStorage.getItem('user');
    return user ? JSON.parse(user) : null;
}

isLoggedIn(): boolean {
    return !!this.getToken();
}

logout(): void {
    if (typeof window !== 'undefined' && typeof localStorage !== 'undefined') {
        localStorage.removeItem('token');
        localStorage.removeItem('user');
    }
    this.authStateSubject.next(null);
}

private readUserFromStorage(): any | null {

```

```

    if (typeof window === 'undefined' || typeof localStorage === 'undefined') {
        return null;
    }

    const user = localStorage.getItem('user');
    return user ? JSON.parse(user) : null;
}
}

```

3. Cart.services.ts

```

4. import { Injectable, inject } from '@angular/core';
5. import { HttpClient } from '@angular/common/http';
6. import { BehaviorSubject, Observable, tap } from 'rxjs';
7. import { environment } from '../../environments/environment';
8. import { Cart } from '../../shared/interfaces/cart.interface';
9. import { PurchaseRequest } from '../../shared/interfaces/purchase-
    request.interface';
10.
11. @Injectable({
12.   providedIn: 'root'
13. })
14. export class CartService {
15.   private http = inject(HttpClient);
16.   private apiUrl = environment.apiUrl;
17.   private cartSubject = new BehaviorSubject<Cart>({ items: [] });
18.
19.   readonly cart$ = this.cartSubject.asObservable();
20.
21.   clearCartState(): void {
22.     this.cartSubject.next({ items: [] });
23.   }
24.
25.   loadCart(): Observable<Cart> {
26.     return this.http.get<Cart>(`${this.apiUrl}/cart`).pipe(
27.       tap((cart) => this.cartSubject.next(cart))
28.     );
29.   }
30.
31.   loadHistory(): Observable<PurchaseRequest[]> {
32.     return
33.     this.http.get<PurchaseRequest[]>(`${this.apiUrl}/cart/history`);

```

```
34.
35. addItem(productId: string, selectedSize: string, quantity = 1) {
36.     return this.http.post<{ message: string; cart: Cart
37.     }>(`${this.apiUrl}/cart/items`, {
38.         product_id: productId,
39.         selected_size: selectedSize,
40.         quantity
41.     }).pipe(
42.         tap((response) => this.cartSubject.next(response.cart))
43.     );
44. }
45. updateItem(productId: string, selectedSize: string, quantity: number) {
46.     return this.http.put<{ message: string; cart: Cart
47.     }>(`${this.apiUrl}/cart/items/${productId}`, {
48.         selected_size: selectedSize,
49.         quantity
50.     }).pipe(
51.         tap((response) => this.cartSubject.next(response.cart))
52.     );
53. }
54. removeItem(productId: string, selectedSize: string) {
55.     return this.http.delete<{ message: string; cart: Cart
56.     }>(`${this.apiUrl}/cart/items/${productId}`, {
57.         body: { selected_size: selectedSize }
58.     }).pipe(
59.         tap((response) => this.cartSubject.next(response.cart))
60.     );
61. }
62. checkout(channel: 'whatsapp' | 'instagram') {
63.     return this.http.post<{ message: string; active_cart: Cart;
64.     submitted_cart: Cart; request: PurchaseRequest
65.     }>(`${this.apiUrl}/cart/checkout`, {
66.         channel
67.     }).pipe(
68.         tap((response) => this.cartSubject.next(response.active_cart))
69.     );
70. }
```

4. contact.services.ts

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { BehaviorSubject, Observable, tap } from 'rxjs';
import { ContactInfo } from '../../shared/interfaces/contact.interface';
import { environment } from '../../environments/environment';

@Injectable({
  providedIn: 'root'
})
export class ContactService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;

  private readonly defaultContact: ContactInfo = {
    whatsapp_number: '',
    whatsapp_label: '',
    facebook: '',
    instagram: '',
    tiktok: '',
    email: '',
    location: ''
  };

  private readonly contactSubject = new
  BehaviorSubject<ContactInfo>(this.defaultContact);
```

```

    readonly contact$ = this.contactSubject.asObservable();

    loadContact(): Observable<ContactInfo> {
        return this.http.get<ContactInfo>(`${this.apiUrl}/contact`).pipe(
            tap((contact) => this.contactSubject.next({ ...this.defaultContact,
...contact })))
    };
}

getCurrentContact(): ContactInfo {
    return this.contactSubject.value;
}

updateContact(payload: ContactInfo) {
    return this.http.put<{ message: string }>(`${this.apiUrl}/contact`,
payload).pipe(
        tap(() => this.contactSubject.next({ ...this.defaultContact, ...payload })))
    };
}
}

```

5. hero.services.ts

```

import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { BehaviorSubject, Observable, tap } from 'rxjs';
import { HeroContent } from '../shared/interfaces/hero.interface';
import { environment } from '../environments/environment';

@Injectable({
    providedIn: 'root'
})
export class HeroService {
    private http = inject(HttpClient);
    private apiUrl = environment.apiUrl;

    private readonly defaultHero: HeroContent = {
        title: 'Bienvenido a ShadowAngels',
        subtitle: 'Descubre nuestros productos, novedades y ofertas especiales.',
        image: ''
    };

    private readonly heroSubject = new
BehaviorSubject<HeroContent>(this.defaultHero);
    readonly hero$ = this.heroSubject.asObservable();
}

```

```

loadHero(): Observable<HeroContent> {
  return this.http.get<HeroContent>(`${this.apiUrl}/hero`).pipe(
    tap((hero) => this.heroSubject.next({ ...this.defaultHero, ...hero }))
  );
}

getCurrentHero(): HeroContent {
  return this.heroSubject.value;
}

updateHero(payload: HeroContent) {
  return this.http.put<{ message: string }>(`${this.apiUrl}/hero`,
payload).pipe(
    tap(() => this.heroSubject.next({ ...this.defaultHero, ...payload }))
  );
}

uploadImage(file: File) {
  const formData = new FormData();
  formData.append('images', file);
  return this.http.post<{ message: string; images: string[]
}>(`${this.apiUrl}/uploads/hero`, formData);
}

getImagePath(image?: string): string {
  if (!image) {
    return '';
  }

  if (image.startsWith('http://') || image.startsWith('https://') ||
image.startsWith('/')) {
    return image;
  }

  return `${environment.uploadBaseUrl}/hero/${image}`;
}
}

```

6. notification.services.ts

```
import { Injectable, inject } from '@angular/core';
```

```

import { HttpClient } from '@angular/common/http';
import { BehaviorSubject, Observable, tap } from 'rxjs';
import { environment } from '../../environments/environment';
import { NotificationItem, UserNotification } from
'../../shared/interfaces/notification.interface';

@Injectable({
  providedIn: 'root'
})
export class NotificationService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;
  private unreadSubject = new BehaviorSubject<number>(0);

  readonly unreadCount$ = this.unreadSubject.asObservable();
  clearUnreadCount(): void {
    this.unreadSubject.next(0);
  }
  getAdminNotifications(): Observable<NotificationItem[]> {
    return
this.http.get<NotificationItem[]>(`${this.apiUrl}/admin/notifications`);
  }
  createNotification(payload: Partial<NotificationItem>) {
    return this.http.post<{ message: string; notification: NotificationItem
}>(`${this.apiUrl}/notifications`, payload);
  }
  updateNotification(id: string, payload: Partial<NotificationItem>) {
    return this.http.put<{ message: string; notification: NotificationItem
}>(`${this.apiUrl}/admin/notifications/${id}`, payload);
  }
  deleteNotification(id: string) {
    return this.http.delete<{ message: string
}>(`${this.apiUrl}/admin/notifications/${id}`);
  }
  uploadImage(file: File) {
    const formData = new FormData();
    formData.append('images', file);
    return this.http.post<{ message: string; images: string[]
}>(`${this.apiUrl}/uploads/notifications`, formData);
  }
  getUserNotifications(): Observable<UserNotification[]> {

```



```

        return
    this.http.get<UserNotification[]>(`${this.apiUrl}/notifications`).pipe(
        tap((notifications) => this.unreadSubject.next(notifications.filter((item)
=> !item.is_read).length))
    );
}
markAsRead(id: string) {
    return this.http.put<{ message: string
}>(`${this.apiUrl}/notifications/${id}/read`, {}).pipe(
        tap(() => this.refreshUnreadCount())
    );
}

deleteUserNotification(id: string) {
    return this.http.delete<{ message: string
}>(`${this.apiUrl}/notifications/${id}`).pipe(
        tap(() => this.refreshUnreadCount())
    );
}

refreshUnreadCount(): void {
    this.getUserNotifications().subscribe();
}
}

```

7. Producto.services.ts

```

8. import { Injectable, inject } from '@angular/core';
9. import { HttpClient } from '@angular/common/http';
10. import { Observable } from 'rxjs';
11. import { environment } from '../../environments/environment';
12. import { Product } from '../../shared/interfaces/product.interface';
13.
14. @Injectable({
15.   providedIn: 'root'
16. })
17. export class ProductService {
18.   private http = inject(HttpClient);
19.   private apiUrl = environment.apiUrl;
20.
21.   getProducts(): Observable<Product[]> {

```

```
22.     return this.http.get<Product[]>(`${this.apiUrl}/products`);
23. }
24.
25. getWomenProducts(): Observable<Product[]> {
26.     return
27.     this.http.get<Product[]>(`${this.apiUrl}/products?category=dama`);
28. }
29. getMenProducts(): Observable<Product[]> {
30.     return
31.     this.http.get<Product[]>(`${this.apiUrl}/products?category=caballero`);
32. }
33. getOffers(): Observable<Product[]> {
34.     return
35.     this.http.get<Product[]>(`${this.apiUrl}/products?offers=true`);
36. }
37. getNews(): Observable<Product[]> {
38.     return
39.     this.http.get<Product[]>(`${this.apiUrl}/products?newest=true`);
40. }
41. getProductById(id: string): Observable<Product> {
42.     return this.http.get<Product>(`${this.apiUrl}/products/${id}`);
43. }
44.
45. createProduct(product: unknown): Observable<unknown> {
46.     return this.http.post(`${this.apiUrl}/products`, product);
47. }
48.
49. updateProduct(id: string, product: unknown): Observable<unknown> {
50.     return this.http.put(`${this.apiUrl}/products/${id}`, product);
51. }
52.
53. deleteProduct(id: string): Observable<unknown> {
54.     return this.http.delete(`${this.apiUrl}/products/${id}`);
55. }
56.
57. createReview(productId: string, payload: { rating: number; comment:
    string }) {
```

```

58.     return this.http.post(`${this.apiUrl}/products/${productId}/reviews`,
    payload);
59.   }
60.
61.   updateReview(reviewId: string, payload: { rating: number; comment:
    string }) {
62.     return this.http.put(`${this.apiUrl}/reviews/${reviewId}`, payload);
63.   }
64.
65.   uploadImages(files: File[]): Observable<{ message: string; images:
    string[] }> {
66.     const formData = new FormData();
67.
68.     files.forEach((file) => formData.append('images', file));
69.
70.     return this.http.post<{ message: string; images: string[] }>(
71.       `${this.apiUrl}/uploads/products`,
72.       formData
73.     );
74.   }
75. }
76.

```

8. profile.services.ts

```

9. import { Injectable, inject } from '@angular/core';
10. import { HttpClient } from '@angular/common/http';
11. import { Observable } from 'rxjs';
12. import { environment } from '../../../environments/environment';
13. import { User } from '../../../shared/interfaces/user.interface';
14.
15. @Injectable({

```

```
16.   providedIn: 'root'
17. })
18. export class ProfileService {
19.   private http = inject(HttpClient);
20.   private apiUrl = environment.apiUrl;
21.
22.   getProfile(): Observable<User> {
23.     return this.http.get<User>(`${this.apiUrl}/profile`);
24.   }
25.
26.   updateProfile(data: Partial<User>): Observable<any> {
27.     return this.http.put(`${this.apiUrl}/profile`, data);
28.   }
29. }
```

9. purchase.services.ts

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
```

```

import { environment } from '../../../environments/environment';
import { PurchaseRequest } from '../../../shared/interfaces/purchase-
request.interface';

@Injectable({
  providedIn: 'root'
})
export class PurchaseRequestService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl;

  createRequest(productId: string, channel: 'whatsapp' | 'instagram',
selectedSize: string) {
    return this.http.post<{ message: string; request: PurchaseRequest }>(
      `${this.apiUrl}/purchase-requests`,
      { product_id: productId, channel, selected_size: selectedSize }
    );
  }

  createGuestRequest(productId: string, channel: 'whatsapp' | 'instagram',
guestName: string, guestContact: string, selectedSize: string) {
    return this.http.post<{ message: string; request: PurchaseRequest }>(
      `${this.apiUrl}/guest-purchase-requests`,
      {
        product_id: productId,
        channel,
        guest_name: guestName,
        guest_contact: guestContact,
        selected_size: selectedSize
      }
    );
  }

  getMyProductStatus(productId: string): Observable<{ request: PurchaseRequest |
null; confirmed_request?: PurchaseRequest | null; can_review: boolean;
has_purchased: boolean }> {
    return this.http.get<{ request: PurchaseRequest | null; confirmed_request?:
PurchaseRequest | null; can_review: boolean; has_purchased: boolean }>(
      `${this.apiUrl}/purchase-requests/status/${productId}`
    );
  }
}

```

```
getAdminRequests(): Observable<PurchaseRequest[]> {  
    return this.http.get<PurchaseRequest[]>(`${this.apiUrl}/admin/purchase-  
requests`);  
}  
  
updateAdminRequest(requestId: string, status: PurchaseRequest['status'],  
adminNote = '') {  
    return this.http.put<{ message: string; request: PurchaseRequest }>(  
        `${this.apiUrl}/admin/purchase-requests/${requestId}`,  
        { status, admin_note: adminNote }  
    );  
}  
  
deleteAdminRequest(requestId: string) {  
    return this.http.delete<{ message: string }>(`${this.apiUrl}/admin/purchase-  
requests/${requestId}`);  
}  
}
```

Modelos

1. Cart_model.py

```
30.from bson import ObjectId
31.from datetime import datetime
32.from models.db import db
33.
34.cart_collection = db["carts"]
35.
36.class CartModel:
37.
38.    @staticmethod
39.    def create_cart(user_id):
40.        cart = {
41.            "user_id": ObjectId(user_id),
42.            "status": "draft",
43.            "items": [],
44.            "request_id": None,
45.            "request_channel": None,
46.            "requested_at": None,
47.            "created_at": datetime.utcnow(),
48.            "updated_at": datetime.utcnow()
49.        }
50.        result = cart_collection.insert_one(cart)
51.        return cart_collection.find_one({"_id": result.inserted_id})
52.
53.    @staticmethod
54.    def get_active_cart(user_id):
55.        cart = cart_collection.find_one(
56.            {
57.                "user_id": ObjectId(user_id),
58.                "status": "draft"
59.            },
60.            sort=[("updated_at", -1)]
61.        )
62.
63.        if not cart:
64.            cart = CartModel.create_cart(user_id)
65.
```

```

66.         return cart
67.
68.     @staticmethod
69.     def get_by_id(cart_id):
70.         try:
71.             return cart_collection.find_one({"_id": ObjectId(cart_id)})
72.         except:
73.             return None
74.
75.     @staticmethod
76.     def get_history(user_id):
77.         return list(cart_collection.find(
78.             {
79.                 "user_id": ObjectId(user_id),
80.                 "status": {"$in": ["pending", "confirmed"]}
81.             }
82.         ).sort("requested_at", -1))
83.
84.     @staticmethod
85.     def add_item(user_id, item):
86.         cart = CartModel.get_active_cart(user_id)
87.         items = cart.get("items", [])
88.
89.         for current_item in items:
90.             if (
91.                 current_item["product_id"] == ObjectId(item["product_id"])
92.                 and
93.                 current_item.get("selected_size") ==
94.                 item.get("selected_size")
95.             ):
96.                 current_item["quantity"] += int(item.get("quantity", 1))
97.                 cart_collection.update_one(
98.                     {"_id": cart["_id"]},
99.                     {"$set": {"items": items, "updated_at":
100.                        datetime.utcnow()}}
101.                 )
102.                 return CartModel.get_active_cart(user_id)
103.
104.                 items.append({
105.                     "product_id": ObjectId(item["product_id"]),
106.                     "product_name": item["product_name"],
107.                     "selected_size": item["selected_size"],

```



```

105.         "product_image": item.get("product_image", ""),
106.         "price": float(item.get("price", 0)),
107.         "final_price": float(item.get("final_price", 0)),
108.         "quantity": int(item.get("quantity", 1))
109.     })
110.
111.     cart_collection.update_one(
112.         {"_id": cart["_id"]},
113.         {"$set": {"items": items, "updated_at":
114.             datetime.utcnow()}}
115.     )
116.     return CartModel.get_active_cart(user_id)
117.
118.     @staticmethod
119.     def update_item_quantity(user_id, product_id, selected_size,
120.                             quantity):
121.         cart = CartModel.get_active_cart(user_id)
122.         items = cart.get("items", [])
123.
124.         for current_item in items:
125.             if (
126.                 current_item["product_id"] == ObjectId(product_id)
127.                 and
128.                 current_item.get("selected_size") == selected_size
129.             ):
130.                 current_item["quantity"] = int(quantity)
131.                 break
132.
133.         cart_collection.update_one(
134.             {"_id": cart["_id"]},
135.             {"$set": {"items": items, "updated_at":
136.                 datetime.utcnow()}}
137.         )
138.         return CartModel.get_active_cart(user_id)
139.
140.     @staticmethod
141.     def remove_item(user_id, product_id, selected_size):
142.         cart = CartModel.get_active_cart(user_id)
143.         items = [
144.             item for item in cart.get("items", [])
145.             if not (
146.                 item["product_id"] == ObjectId(product_id) and

```

```
143.         item.get("selected_size") == selected_size
144.     )
145. ]
146.
147.     cart_collection.update_one(
148.         {"_id": cart["_id"]},
149.         {"$set": {"items": items, "updated_at":
datetime.utcnow()}}
150.     )
151.     return CartModel.get_active_cart(user_id)
152.
153.     @staticmethod
154.     def submit_cart(cart_id, request_id, channel):
155.         cart = CartModel.get_by_id(cart_id)
156.         if not cart:
157.             return None
158.
159.         cart_collection.update_one(
160.             {"_id": cart["_id"]},
161.             {"$set": {
162.                 "status": "pending",
163.                 "request_id": ObjectId(request_id),
164.                 "request_channel": channel,
165.                 "requested_at": datetime.utcnow(),
166.                 "updated_at": datetime.utcnow()
167.             }}
168.         )
169.         return CartModel.get_by_id(cart_id)
170.
171.     @staticmethod
172.     def sync_request_status(cart_id, status):
173.         if not cart_id:
174.             return None
175.
176.         return cart_collection.update_one(
177.             {"_id": ObjectId(cart_id)},
178.             {"$set": {
179.                 "status": status,
180.                 "updated_at": datetime.utcnow()
181.             }}
182.         )
183.
```

2. contact_model.py

```
from models.db import db

contact_collection = db["contact"]

class ContactModel:

    @staticmethod
    def get_contact():
        contact = contact_collection.find_one({})
        if not contact:
            default_contact = {
                "whatsapp_number": "",
                "whatsapp_label": "",
                "facebook": "",
                "instagram": "",
                "tiktok": "",
                "email": "",
                "location": ""
            }
            contact_collection.insert_one(default_contact)
            return contact_collection.find_one({})
        return contact

    @staticmethod
    def update_contact(data):
        current = ContactModel.get_contact()
        return contact_collection.update_one(
            {"_id": current["_id"]},
            {"$set": {
                "whatsapp_number": data.get("whatsapp_number", "").strip(),
                "whatsapp_label": data.get("whatsapp_label", "").strip(),
                "facebook": data.get("facebook", ""),
                "instagram": data.get("instagram", ""),
                "tiktok": data.get("tiktok", ""),
                "email": data.get("email", ""),
                "location": data.get("location", "")
            }},
            upsert=True
        )
    )
```

3. db.py

```
import os

import certifi
from pymongo import MongoClient
from pymongo.server_api import ServerApi

from config import Config

_client: MongoClient | None = None
_client_pid: int | None = None

def _as_bool(value: str | None) -> bool:
    return str(value or "").strip().lower() in {"1", "true", "yes", "on"}

def _build_client() -> MongoClient:
    uri = Config.MONGO_URI
    client_options = {
        "serverSelectionTimeoutMS": 30000,
        "connectTimeoutMS": 20000,
        "socketTimeoutMS": 20000,
        "retryWrites": True,
        "appName": "ShadowAngels",
        "server_api": ServerApi("1"),
    }

    # Atlas / TLS-enabled deployments should use certifi's CA bundle.
    if uri.startswith("mongodb+srv://") or "ssl=true" in uri.lower() or
"tls=true" in uri.lower():
        client_options["tlsCAFile"] = certifi.where()

    # Optional escape hatch for deployment diagnostics only.
    if _as_bool(os.getenv("MONGO_TLS_ALLOW_INVALID_CERTIFICATES")):
        client_options["tlsAllowInvalidCertificates"] = True

    return MongoClient(uri, **client_options)

def get_client() -> MongoClient:
    global _client, _client_pid
```

```
current_pid = os.getpid()
if _client is None or _client_pid != current_pid:
    _client = _build_client()
    _client_pid = current_pid

return _client

def get_db():
    return get_client()[Config.DB_NAME]

class CollectionProxy:
    def __init__(self, name: str):
        self.name = name

    def _collection(self):
        return get_db()[self.name]

    def __getattr__(self, item):
        return getattr(self._collection(), item)

class DatabaseProxy:
    def __getitem__(self, name: str) -> CollectionProxy:
        return CollectionProxy(name)

    def __getattr__(self, item):
        return getattr(get_db(), item)

db = DatabaseProxy()
```

4. hero_model.py

```
import os

import certifi
from pymongo import MongoClient
from pymongo.server_api import ServerApi

from config import Config

_client: MongoClient | None = None
_client_pid: int | None = None

def _as_bool(value: str | None) -> bool:
    return str(value or "").strip().lower() in {"1", "true", "yes", "on"}

def _build_client() -> MongoClient:
    uri = Config.MONGO_URI
    client_options = {
        "serverSelectionTimeoutMS": 30000,
        "connectTimeoutMS": 20000,
        "socketTimeoutMS": 20000,
        "retryWrites": True,
        "appName": "ShadowAngels",
        "server_api": ServerApi("1"),
    }

    # Atlas / TLS-enabled deployments should use certifi's CA bundle.
    if uri.startswith("mongodb+srv://") or "ssl=true" in uri.lower() or
"tls=true" in uri.lower():
        client_options["tlsCAFile"] = certifi.where()

    # Optional escape hatch for deployment diagnostics only.
    if _as_bool(os.getenv("MONGO_TLS_ALLOW_INVALID_CERTIFICATES")):
        client_options["tlsAllowInvalidCertificates"] = True

    return MongoClient(uri, **client_options)

def get_client() -> MongoClient:
    global _client, _client_pid
```

```
current_pid = os.getpid()
if _client is None or _client_pid != current_pid:
    _client = _build_client()
    _client_pid = current_pid

return _client

def get_db():
    return get_client()[Config.DB_NAME]

class CollectionProxy:
    def __init__(self, name: str):
        self.name = name

    def _collection(self):
        return get_db()[self.name]

    def __getattr__(self, item):
        return getattr(self._collection(), item)

class DatabaseProxy:
    def __getitem__(self, name: str) -> CollectionProxy:
        return CollectionProxy(name)

    def __getattr__(self, item):
        return getattr(get_db(), item)

db = DatabaseProxy()
```

5. notification.model.py

```
from bson import ObjectId
from datetime import datetime
from models.db import db

notifications_collection = db["notifications"]
user_notifications_collection = db["user_notifications"]
users_collection = db["users"]

class NotificationModel:

    @staticmethod
    def create_notification(data):
        status = data.get("status", "draft")
        scheduled_for = data.get("scheduled_for")

        notification = {
            "title": data.get("title", "").strip(),
            "summary": data.get("summary", "").strip(),
            "content": data.get("content", "").strip(),
            "image": data.get("image", "").strip(),
            "status": status,
            "created_by": ObjectId(data["created_by"]),
            "created_at": datetime.utcnow(),
            "scheduled_for": scheduled_for,
            "published_at": datetime.utcnow() if status == "published" else None
        }
        result = notifications_collection.insert_one(notification)
        notification = notifications_collection.find_one({"_id":
result.inserted_id})

        if status == "published":
            NotificationModel.assign_to_users(notification)

        return notification

    @staticmethod
    def get_all():
        NotificationModel.publish_due_notifications()
        return list(notifications_collection.find({
            "is_system": {"$ne": True}
```



```

    }).sort("created_at", -1))

    @staticmethod
    def get_by_id(notification_id):
        try:
            return notifications_collection.find_one({"_id":
ObjectId(notification_id)})
        except:
            return None

    @staticmethod
    def update_notification(notification_id, data):
        current = NotificationModel.get_by_id(notification_id)
        if not current:
            return None

        next_status = data.get("status", current.get("status", "draft"))
        if current.get("status") == "published" and next_status != "published":
            next_status = "published"

        update_fields = {
            "title": data.get("title", current.get("title", "")).strip(),
            "summary": data.get("summary", current.get("summary", "")).strip(),
            "content": data.get("content", current.get("content", "")).strip(),
            "image": data.get("image", current.get("image", "")).strip(),
            "status": next_status,
            "scheduled_for": data.get("scheduled_for",
current.get("scheduled_for")),
        }

        if next_status == "published" and not current.get("published_at"):
            update_fields["published_at"] = datetime.utcnow()

        notifications_collection.update_one(
            {"_id": ObjectId(notification_id)},
            {"$set": update_fields}
        )

        updated = NotificationModel.get_by_id(notification_id)

        if next_status == "published" and current.get("status") != "published":
            NotificationModel.assign_to_users(updated)

```

```

        return updated

    @staticmethod
    def delete_notification(notification_id):
        user_notifications_collection.delete_many({"notification_id":
ObjectId(notification_id)})
        return notifications_collection.delete_one({"_id":
ObjectId(notification_id)})

    @staticmethod
    def publish_due_notifications():
        now = datetime.utcnow().isoformat()
        due_notifications = list(notifications_collection.find({
            "status": "scheduled",
            "scheduled_for": {"$lte": now}
        )))

        for notification in due_notifications:
            notifications_collection.update_one(
                {"_id": notification["_id"]},
                {"$set": {"status": "published", "published_at":
datetime.utcnow()}}
            )
            published = NotificationModel.get_by_id(str(notification["_id"]))
            NotificationModel.assign_to_users(published)

    @staticmethod
    def assign_to_users(notification):
        if not notification:
            return

        users = list(users_collection.find({"role": "user", "is_active": True}))
        bulk_docs = []

        for user in users:
            exists = user_notifications_collection.find_one({
                "notification_id": notification["_id"],
                "user_id": user["_id"]
            })
            if exists:
                continue

```

```

        bulk_docs.append({
            "notification_id": notification["_id"],
            "user_id": user["_id"],
            "is_deleted": False,
            "is_read": False,
            "assigned_at": datetime.utcnow()
        })

    if bulk_docs:
        user_notifications_collection.insert_many(bulk_docs)

    @staticmethod
    def assign_to_specific_users(notification, user_ids):
        if not notification or not user_ids:
            return

        bulk_docs = []

        for user_id in user_ids:
            object_id = ObjectId(user_id)
            exists = user_notifications_collection.find_one({
                "notification_id": notification["_id"],
                "user_id": object_id
            })
            if exists:
                continue

            bulk_docs.append({
                "notification_id": notification["_id"],
                "user_id": object_id,
                "is_deleted": False,
                "is_read": False,
                "assigned_at": datetime.utcnow()
            })

        if bulk_docs:
            user_notifications_collection.insert_many(bulk_docs)

    @staticmethod
    def create_system_notification_for_users(data, user_ids):
        notification = {

```

```

        "title": data.get("title", "").strip(),
        "summary": data.get("summary", "").strip(),
        "content": data.get("content", "").strip(),
        "image": data.get("image", "").strip(),
        "target_type": data.get("target_type"),
        "target_id": data.get("target_id"),
        "related_items": data.get("related_items", []),
        "status": "published",
        "created_by": ObjectId(data["created_by"]),
        "created_at": datetime.utcnow(),
        "scheduled_for": None,
        "published_at": datetime.utcnow(),
        "is_system": True
    }
    result = notifications_collection.insert_one(notification)
    created = notifications_collection.find_one({"_id": result.inserted_id})
    NotificationModel.assign_to_specific_users(created, user_ids)
    return created

@staticmethod
def get_user_notifications(user_id):
    NotificationModel.publish_due_notifications()
    pipeline = [
        {
            "$match": {
                "user_id": ObjectId(user_id),
                "is_deleted": False
            }
        },
        {
            "$lookup": {
                "from": "notifications",
                "localField": "notification_id",
                "foreignField": "_id",
                "as": "notification"
            }
        },
        {"$unwind": "$notification"},
        {"$sort": {"notification.published_at": -1,
"notification.created_at": -1}}
    ]
    return list(user_notifications_collection.aggregate(pipeline))

```

```
@staticmethod
def mark_as_read(user_id, user_notification_id):
    return user_notifications_collection.update_one(
        {
            "_id": ObjectId(user_notification_id),
            "user_id": ObjectId(user_id)
        },
        {"$set": {"is_read": True}}
    )

@staticmethod
def delete_user_notification(user_id, user_notification_id):
    return user_notifications_collection.update_one(
        {
            "_id": ObjectId(user_notification_id),
            "user_id": ObjectId(user_id)
        },
        {"$set": {"is_deleted": True}}
    )
```

6. product.model.py

```
import os
from bson import ObjectId
from datetime import datetime
from models.db import db

products_collection = db["products"]

class ProductModel:

    @staticmethod
    def create_product(data):
        price = float(data.get("price", 0))
        discount = float(data.get("discount", 0))
        final_price = round(price - (price * discount / 100), 2)

        product = {
            "name": data.get("name", "").strip(),
            "description": data.get("description", "").strip(),
            "category": data.get("category", "").strip().lower(), # dama,
            "sizes": data.get("sizes", []), # ["S","M","L"]
            "price": price,
            "discount": discount,
            "final_price": final_price,
            "images": data.get("images", []),
            "is_new": data.get("is_new", False),
            "is_active": True,
            "created_at": datetime.utcnow()
        }
        return products_collection.insert_one(product)

    @staticmethod
    def get_all(filters=None):
        query = {"is_active": True}
        if filters:
            query.update(filters)
        return list(products_collection.find(query))

    @staticmethod
    def get_by_id(product_id):
```

```

        try:
            return products_collection.find_one({"_id": ObjectId(product_id),
"is_active": True})
        except:
            return None

    @staticmethod
    def update_product(product_id, data):
        update_fields = {}

        for field in ["name", "description", "category", "sizes", "images",
"is_new", "is_active"]:
            if field in data:
                update_fields[field] = data[field]

        if "price" in data or "discount" in data:
            current = ProductModel.get_by_id(product_id)
            if not current:
                return None

            price = float(data.get("price", current.get("price", 0)))
            discount = float(data.get("discount", current.get("discount", 0)))
            update_fields["price"] = price
            update_fields["discount"] = discount
            update_fields["final_price"] = round(price - (price * discount /
100), 2)

        if not update_fields:
            return None

        return products_collection.update_one(
            {"_id": ObjectId(product_id)},
            {"$set": update_fields}
        )

    @staticmethod
    def delete_product(product_id):
        return products_collection.update_one(
            {"_id": ObjectId(product_id)},
            {"$set": {"is_active": False, "images": []}}
        )

```

```
@staticmethod
def get_by_image_name(image_name, exclude_product_id=None):
    query = {"images": image_name}

    if exclude_product_id:
        query["_id"] = {"$ne": ObjectId(exclude_product_id)}

    return products_collection.find_one(query)

@staticmethod
def cleanup_unused_images(images, upload_folder, exclude_product_id=None):
    for image_name in images:
        if not image_name:
            continue

        if ProductModel.get_by_image_name(image_name, exclude_product_id):
            continue

        image_path = os.path.join(upload_folder, image_name)
        if os.path.exists(image_path):
            os.remove(image_path)
```


7. purchase_request_model.py

```
from bson import ObjectId
from datetime import datetime
from models.db import db

purchase_requests_collection = db["purchase_requests"]

class PurchaseRequestModel:

    @staticmethod
    def create_or_get_pending(data):
        query = {"status": "pending"}
        if data.get("user_id"):
            query["user_id"] = ObjectId(data["user_id"])
            query["product_id"] = ObjectId(data["product_id"])
            query["selected_size"] = data["selected_size"]
        else:
            query["guest_contact"] = data["guest_contact"].strip()
            query["product_id"] = ObjectId(data["product_id"])
            query["selected_size"] = data["selected_size"]

        existing = purchase_requests_collection.find_one({
            **query
        })

        if existing:
            return existing, False

        purchase_request = {
            "user_id": ObjectId(data["user_id"]) if data.get("user_id") else
None,
            "user_name": data["user_name"],
            "user_email": data.get("user_email", ""),
            "is_guest": not bool(data.get("user_id")),
            "guest_contact": data.get("guest_contact", "").strip(),
            "request_type": "single",
            "product_id": ObjectId(data["product_id"]),
            "product_name": data["product_name"],
            "selected_size": data["selected_size"],
            "channel": data["channel"],
            "status": "pending",
```

```

        "admin_note": "",
        "created_at": datetime.utcnow(),
        "updated_at": datetime.utcnow(),
        "confirmed_at": None,
        "confirmed_by": None
    }

    result = purchase_requests_collection.insert_one(purchase_request)
    created = purchase_requests_collection.find_one({"_id":
result.inserted_id})
    return created, True

    @staticmethod
    def create_from_cart(data):
        purchase_request = {
            "user_id": ObjectId(data["user_id"]),
            "user_name": data["user_name"],
            "user_email": data["user_email"],
            "request_type": "cart",
            "cart_id": ObjectId(data["cart_id"]),
            "items": [
                {
                    "product_id": ObjectId(item["product_id"]),
                    "product_name": item["product_name"],
                    "selected_size": item["selected_size"],
                    "product_image": item.get("product_image", ""),
                    "price": float(item.get("price", 0)),
                    "final_price": float(item.get("final_price", 0)),
                    "quantity": int(item.get("quantity", 1))
                }
                for item in data["items"]
            ],
            "channel": data["channel"],
            "status": "pending",
            "admin_note": "",
            "created_at": datetime.utcnow(),
            "updated_at": datetime.utcnow(),
            "confirmed_at": None,
            "confirmed_by": None
        }

        result = purchase_requests_collection.insert_one(purchase_request)

```

```

        return purchase_requests_collection.find_one({"_id": result.inserted_id})

    @staticmethod
    def get_by_cart_id(cart_id):
        try:
            return purchase_requests_collection.find_one(
                {"cart_id": ObjectId(cart_id)},
                sort=[("created_at", -1)]
            )
        except:
            return None

    @staticmethod
    def get_all():
        return list(purchase_requests_collection.find({}).sort("created_at", -1))

    @staticmethod
    def get_user_history(user_id):
        return list(purchase_requests_collection.find({
            "user_id": ObjectId(user_id),
            "status": {"$in": ["pending", "confirmed"]}
        }).sort("created_at", -1))

    @staticmethod
    def get_by_id(request_id):
        try:
            return purchase_requests_collection.find_one({"_id":
ObjectId(request_id)})
        except:
            return None

    @staticmethod
    def delete(request_id):
        try:
            return purchase_requests_collection.delete_one({"_id":
ObjectId(request_id)})
        except:
            return None

    @staticmethod
    def update_status(request_id, status, admin_id, admin_note=""):
        update_fields = {

```

```

        "status": status,
        "admin_note": admin_note,
        "updated_at": datetime.utcnow(),
        "confirmed_by": ObjectId(admin_id)
    }

    if status == "confirmed":
        update_fields["confirmed_at"] = datetime.utcnow()
    else:
        update_fields["confirmed_at"] = None

    return purchase_requests_collection.update_one(
        {"_id": ObjectId(request_id)},
        {"$set": update_fields}
    )

@staticmethod
def get_user_product_status(user_id, product_id):
    try:
        confirmed = purchase_requests_collection.find_one(
            {
                "user_id": ObjectId(user_id),
                "status": "confirmed",
                "$or": [
                    {"product_id": ObjectId(product_id)},
                    {"items.product_id": ObjectId(product_id)}
                ]
            },
            sort=[("confirmed_at", -1), ("created_at", -1)]
        )

        pending = purchase_requests_collection.find_one(
            {
                "user_id": ObjectId(user_id),
                "status": "pending",
                "$or": [
                    {"product_id": ObjectId(product_id)},
                    {"items.product_id": ObjectId(product_id)}
                ]
            },
            sort=[("created_at", -1)]
        )

```

```

        return {
            "confirmed_request": confirmed,
            "latest_request": pending or confirmed
        }
    except:
        return {
            "confirmed_request": None,
            "latest_request": None
        }

    @staticmethod
    def can_review(user_id, product_id):
        return purchase_requests_collection.find_one({
            "user_id": ObjectId(user_id),
            "status": "confirmed",
            "$or": [
                {"product_id": ObjectId(product_id)},
                {"items.product_id": ObjectId(product_id)}
            ]
        })

    @staticmethod
    def has_purchased(user_id, product_id):
        return bool(PurchaseRequestModel.can_review(user_id, product_id))

```

8. review.model.py

```
from bson import ObjectId
from datetime import datetime, timedelta
from models.db import db

reviews_collection = db["reviews"]
products_collection = db["products"]

class ReviewModel:

    @staticmethod
    def already_reviewed(product_id, user_id):
        return reviews_collection.find_one({
            "product_id": ObjectId(product_id),
            "user_id": ObjectId(user_id)
        })

    @staticmethod
    def create_review(data):
        review = {
            "product_id": ObjectId(data["product_id"]),
            "user_id": ObjectId(data["user_id"]),
            "user_name": data["user_name"],
            "rating": int(data["rating"]),
            "comment": data["comment"].strip(),
            "created_at": datetime.utcnow()
        }
        return reviews_collection.insert_one(review)

    @staticmethod
    def get_by_product(product_id):
        return list(reviews_collection.find(
            {"product_id": ObjectId(product_id)}
        ).sort("created_at", -1))

    @staticmethod
    def get_all():
        return list(reviews_collection.find({}).sort("created_at", -1))

    @staticmethod
    def enrich_with_product(review):
```

```

        if not review:
            return None

        product = products_collection.find_one({"_id": review.get("product_id")})
        enriched = dict(review)
        enriched["product_name"] = product.get("name") if product else "Producto
eliminado"
        enriched["product_image"] = product.get("images", [""])[0] if product and
product.get("images") else ""
        return enriched

    @staticmethod
    def get_by_id(review_id):
        try:
            return reviews_collection.find_one({"_id": ObjectId(review_id)})
        except:
            return None

    @staticmethod
    def can_edit(review):
        if not review:
            return False
        created_at = review.get("created_at")
        if not created_at:
            return False
        return datetime.utcnow() <= created_at + timedelta(minutes=10)

    @staticmethod
    def update_review(review_id, data):
        return reviews_collection.update_one(
            {"_id": ObjectId(review_id)},
            {
                "$set": {
                    "rating": int(data["rating"]),
                    "comment": data["comment"].strip(),
                    "updated_at": datetime.utcnow()
                }
            }
        )

    @staticmethod
    def delete_review(review_id):
        return reviews_collection.delete_one({"_id": ObjectId(review_id)})

```

9. user_model.py

```
from bson import ObjectId
from werkzeug.security import generate_password_hash, check_password_hash
from models.db import db

users_collection = db["users"]

class UserModel:

    @staticmethod
    def create_user(data):
        user = {
            "name": data.get("name", "").strip(),
            "email": data.get("email", "").strip().lower(),
            "password": generate_password_hash(data.get("password", "")),
            "role": data.get("role", "user"), # user, admin, superadmin
            "is_active": True,
            "created_at": data.get("created_at"),
        }
        return users_collection.insert_one(user)

    @staticmethod
    def find_by_email(email):
        return users_collection.find_one({"email": email.strip().lower()})

    @staticmethod
    def find_by_email_except(email, user_id):
        return users_collection.find_one({
            "email": email.strip().lower(),
            "_id": {"$ne": ObjectId(user_id)}
        })

    @staticmethod
    def find_by_id(user_id):
        try:
            return users_collection.find_one({"_id": ObjectId(user_id)})
        except:
            return None

    @staticmethod
    def get_all_admins():
```



```

        return list(users_collection.find(
            {"role": {"$in": ["admin", "superadmin"]}},
            {"password": 0}
        ))

    @staticmethod
    def update_user(user_id, data):
        update_fields = {}

        if "name" in data:
            update_fields["name"] = data["name"].strip()

        if "email" in data:
            update_fields["email"] = data["email"].strip().lower()

        if "password" in data and data["password"]:
            update_fields["password"] = generate_password_hash(data["password"])

        if "role" in data:
            update_fields["role"] = data["role"]

        if not update_fields:
            return None

        return users_collection.update_one(
            {"_id": ObjectId(user_id)},
            {"$set": update_fields}
        )

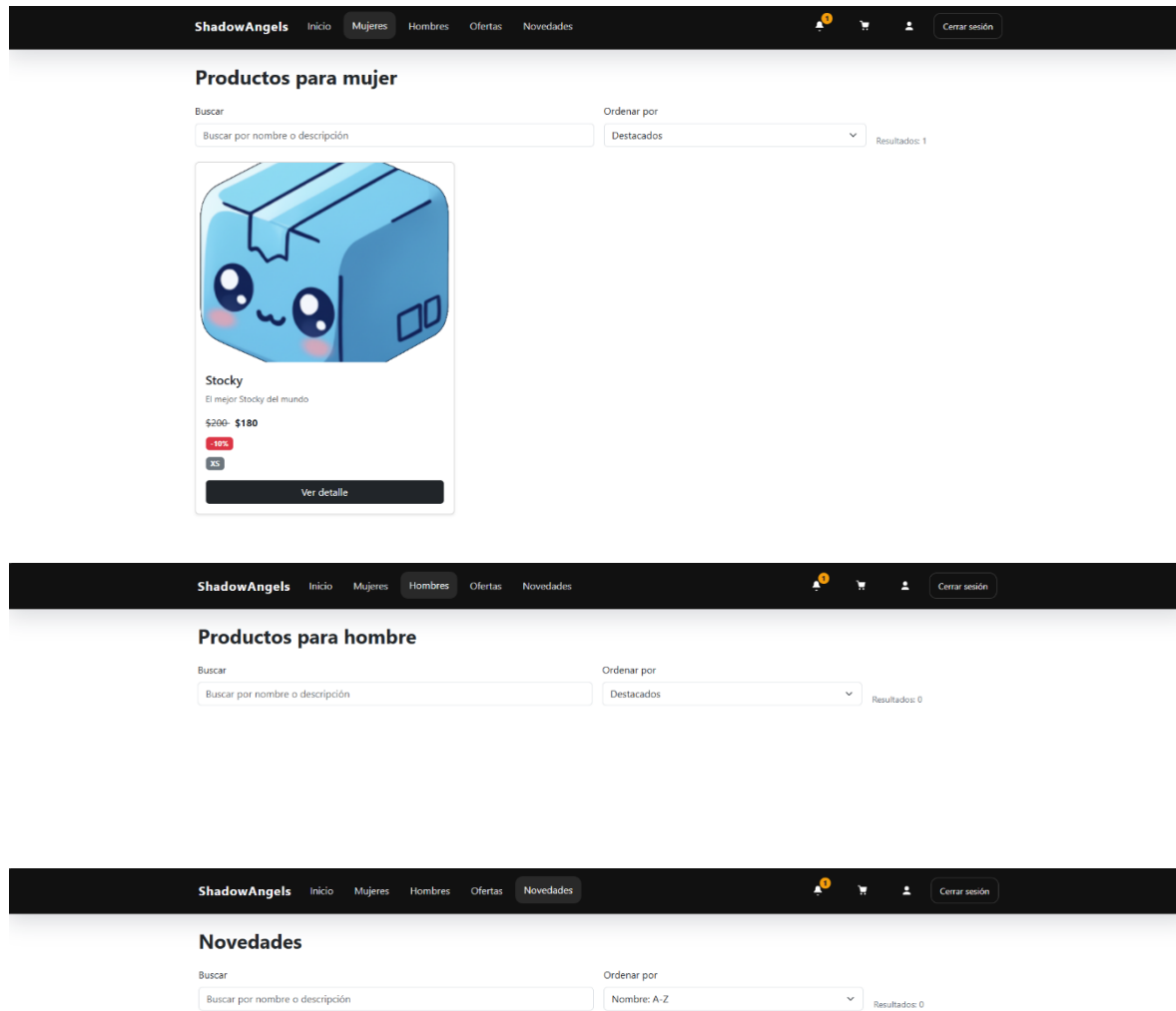
    @staticmethod
    def delete_user(user_id):
        return users_collection.delete_one({"_id": ObjectId(user_id)})

    @staticmethod
    def verify_password(user, password):
        return check_password_hash(user["password"], password)

```

Captura app.py

Capturas Página



Ofertas

Buscar

Ordenar por

Buscar por nombre o descripción

Mayor descuento

Resultados: 1



Stocky

El mejor Stocky del mundo

~~\$200~~ **\$180**

10%

XS

Ver detalle

Notificaciones

Avisos publicados por la administración.

Nuevas

Leídas

Compra confirmada

15 abr 2026, 03:14 p.m.

Tu solicitud de compra fue confirmada. Ya puedes dejar reseñas en los productos incluidos.

Ver

Ver carrito

Marcar como leída

Eliminar

Nuevo

Tu carrito

Aquí puedes ajustar cantidades, quitar productos y enviar una sola solicitud.

Seguir viendo productos

Tu carrito está vacío

Agrega productos desde el detalle para empezar tu solicitud.

Historial de compras

Se muestran solo solicitudes pendientes y compras confirmadas.

Todas

Pendientes

Confirmadas

Carrito con 3 prendas

Apr 15, 2026

Stocky - talla XS x3

Compra confirmada. Ya puedes dejar reseñas en los productos de este carrito.

CONFIRMED

\$540

Total: \$540

Gestion de perfil

CUENTA

Usuario

Activa

Mi perfil

Revisa tu información personal y actualiza los datos editables de tu cuenta.

Nombre completo

Luis

4/90

Este nombre se muestra en tus reseñas y solicitudes.

Correo electrónico

angel@gmail.com

15/50

Usamos este correo para identificar tu cuenta.

Rol

Usuario

Estado de cuenta

Activa

ID de usuario

69dfaa589113aa753d3ff9cb

Recargar datos

Guardar cambios

Crear cuenta

Nombre

0/20

Correo

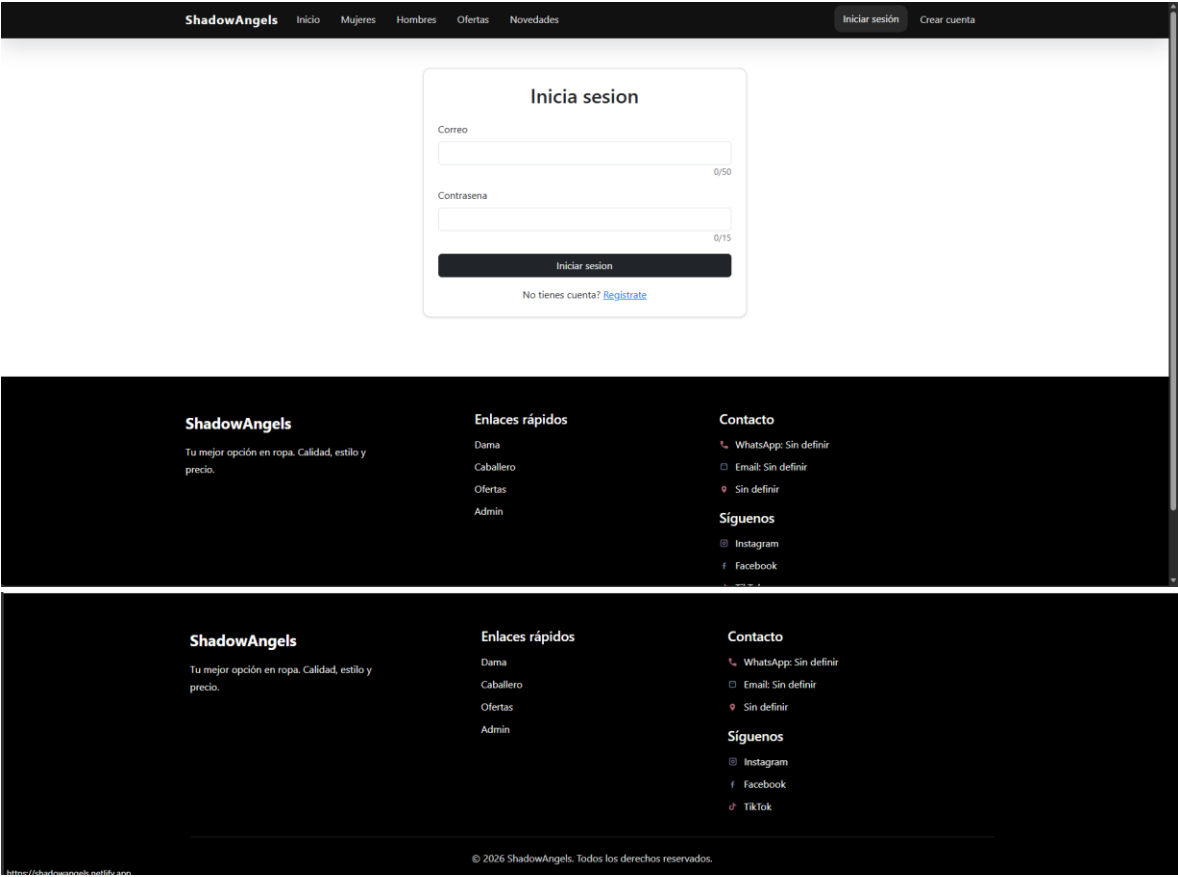
0/50

Contraseña

0/15

Crear cuenta

Ya tienes cuenta? [Inicia sesión](#)



Productos destacados

Explora nuestro catálogo

Actualizar

Buscar

Buscar por nombre o descripción

Ordenar por

Destacados

Resultados: 1



Stocky

El mejor Stocky del mundo

~~\$200~~ **\$100**

10%

8%

Ver detalle

ShadowAngels

Inicio

Mujeres

Hombres

Ofertas

Novedades



Cerrar sesión

Bienvenido a ShadowAngels

Descubre nuestros productos, novedades y ofertas especiales.



Conclusión

El presente documento establece las bases formales del proyecto ShadowAngels, definiendo con claridad el contexto del negocio, la problemática identificada, el alcance del sistema, los perfiles de usuario y las funcionalidades esperadas para cada uno de ellos, así como la estructura inicial propuesta para el desarrollo del frontend.

A lo largo del proceso de documentación se identificaron y resolvieron aspectos importantes de lógica y estructura del sistema, entre los que destacan la definición de cuatro perfiles de usuario diferenciados, el establecimiento de una jerarquía de roles entre administrador general y super administrador, la clarificación del carácter opcional del registro de usuarios, el diseño del sistema de notificaciones interno y las restricciones operativas aplicadas al rol de mayor jerarquía para garantizar la integridad del sistema.

El sistema a desarrollar responde a una necesidad real de una clienta cuyo negocio carece de presencia digital formal, por lo que ShadowAngels representa una solución concreta, acotada y funcional que permitirá al negocio contar con una vitrina digital desde la cual sus clientes podrán explorar el inventario disponible de manera autónoma, mientras que los administradores podrán gestionar dicho contenido de forma sencilla e intuitiva.

Es importante destacar que la estructura y el alcance definidos en este documento corresponden a la versión inicial del proyecto y podrán estar sujetos a ajustes menores conforme avance el proceso de desarrollo, siempre dentro de los límites establecidos en la sección de alcance y en concordancia con los acuerdos establecidos con la clienta.